

# Introduction to Neural Networks

Elizabeth Heon

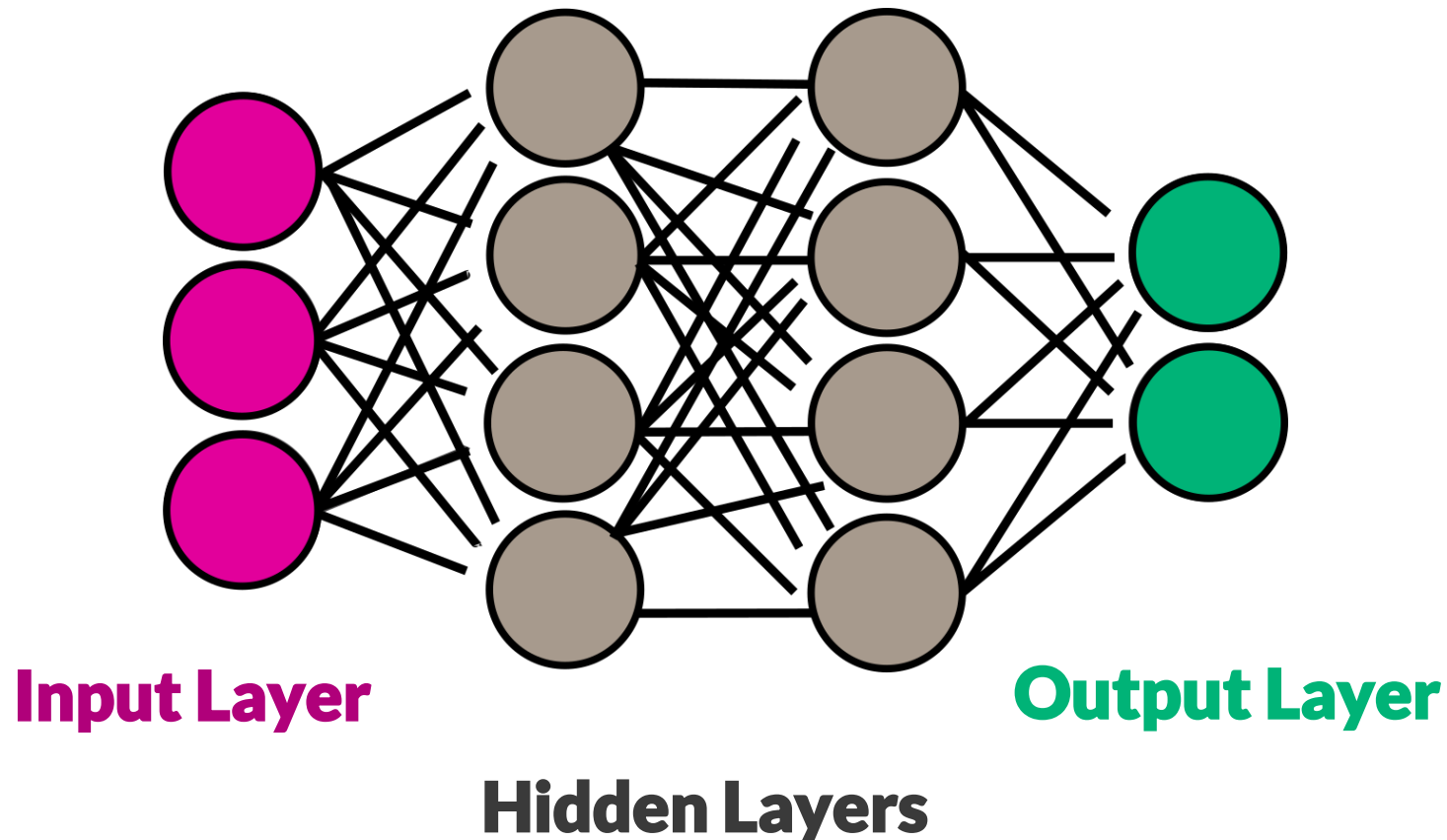
6/22/2026

# INTRO

*What, where, why?*

# What is a Neural Network?

A neural network is a machine learning model formed of interconnected layers of nodes, called **neurons**, which are designed to mimic the function of biological neurons



# Types of Learning

**Supervised Learning:** Requires a set of **labelled training data**, from which the network learns to predict for new data.

- Regression
- Classification



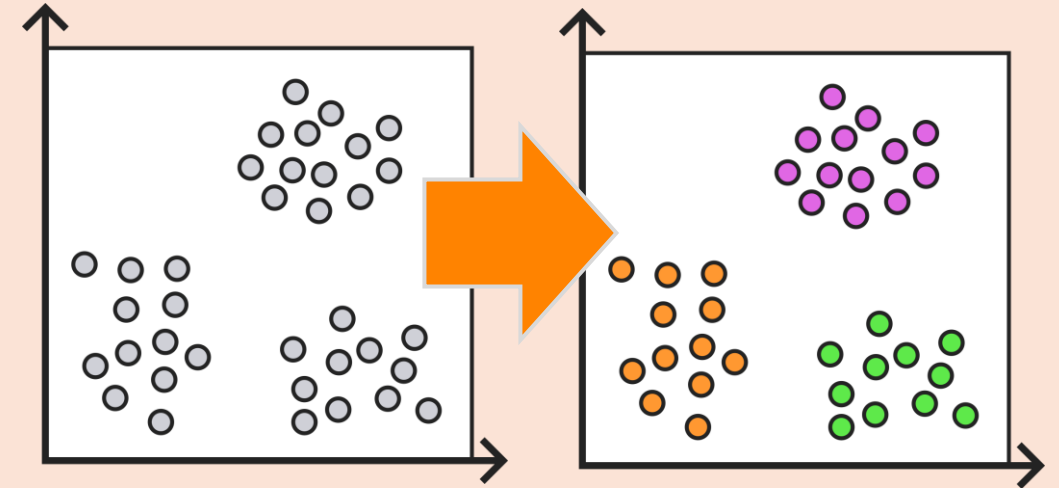
**LABEL = DOG**



**LABEL = DIAGNOSIS**

**Unsupervised Learning:** No labelled training data is provided – network learns intrinsic patterns in the data

- Clustering



**K-MEANS CLUSTERING**

**This talk will focus on supervised learning, where a network is supplied labelled training data**

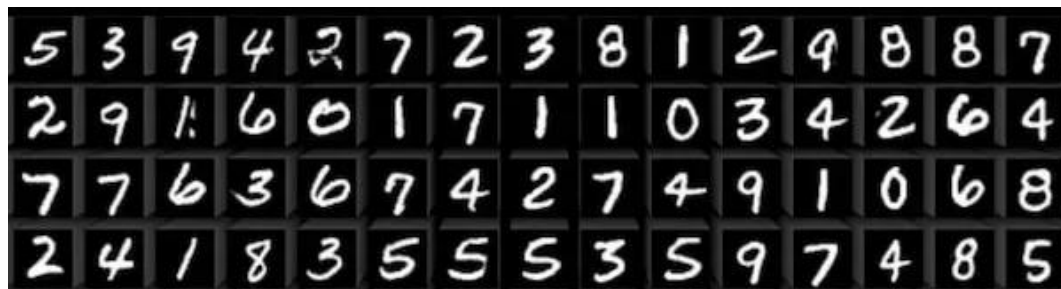


# Regression and Classification

5

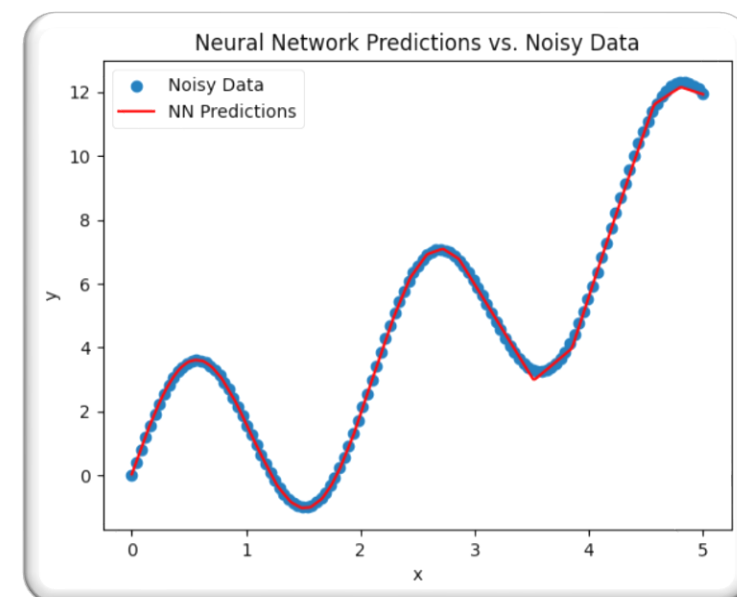
## Classification

- Output of network is **categorical**
- Examples:
  - Character Recognition:
    - Input: Image of handwritten character,
    - Output: Letter A-Z or digit 0-9



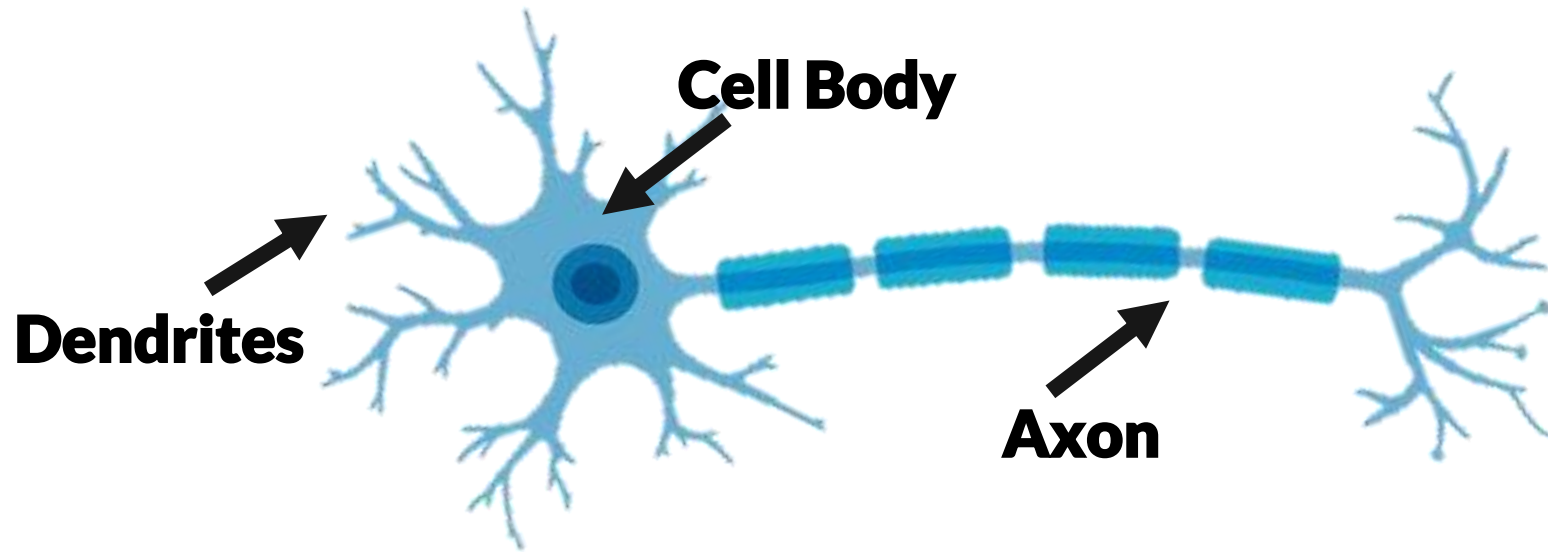
## Regression

- Output of network is **numerical**
- Examples:
  - Linear regression / function interpolation



# I. NEURONS

# Biological Neurons:



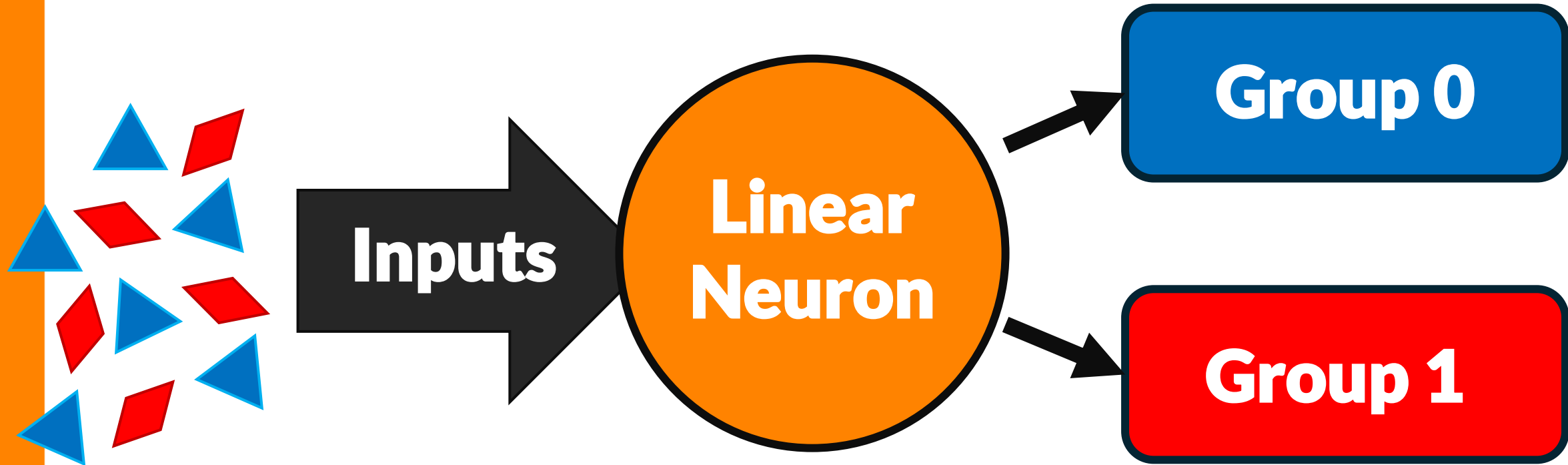
- Dendrites receive messages from other neurons – these messages can be excitatory (encourage the neuron to fire) or inhibitory (prevent firing)
- Depending on the combination of inputs, the neuron may fire an “action potential” which travels down the axon to other neurons

**A (biological) neuron takes a combination of inputs and decides which output to produce – potential or no potential**



# Artificial Neurons

- Consider a **binary linear neuron** - given a set of inputs, the neuron can output either 0 or 1.



- **Depending on the inputs it receives, the neuron “chooses” to output either 0 or 1.**
  - These neurons classify inputs into one of two categories





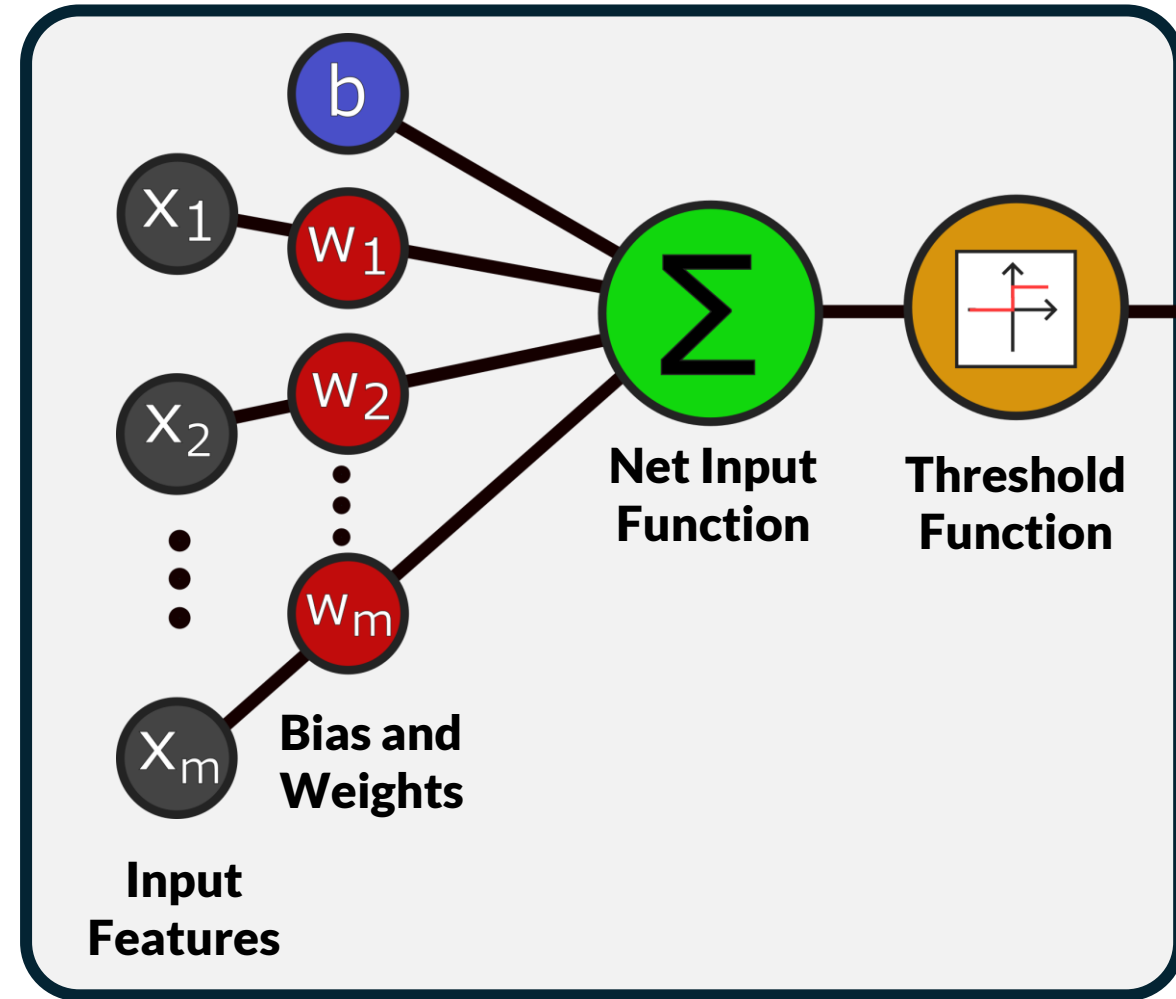
# Artificial Neuron Structure

- 1) Neuron receives a vector of inputs,  $x_i$
- 2) Inputs are multiplied by vector of **weights** to form **net input function**:

$$z = \sum_i x_i w_i$$

- 3) The net input is compared to a value called the **threshold** ( $\theta$ ):

$$\text{Output: } \sigma(z) = \begin{cases} 1 & \text{if } z > \theta \\ 0 & \text{otherwise} \end{cases}$$



Artificial neurons take in a set of **inputs** and determine what output to produce using a set of **weights** and a **threshold**



# Bias

- The critical point between inputs that output 0 and inputs that output 1 is given by:

$$z = \sum_i x_i w_i = \theta$$

- Subtract the value of  $\theta$  from both sides:

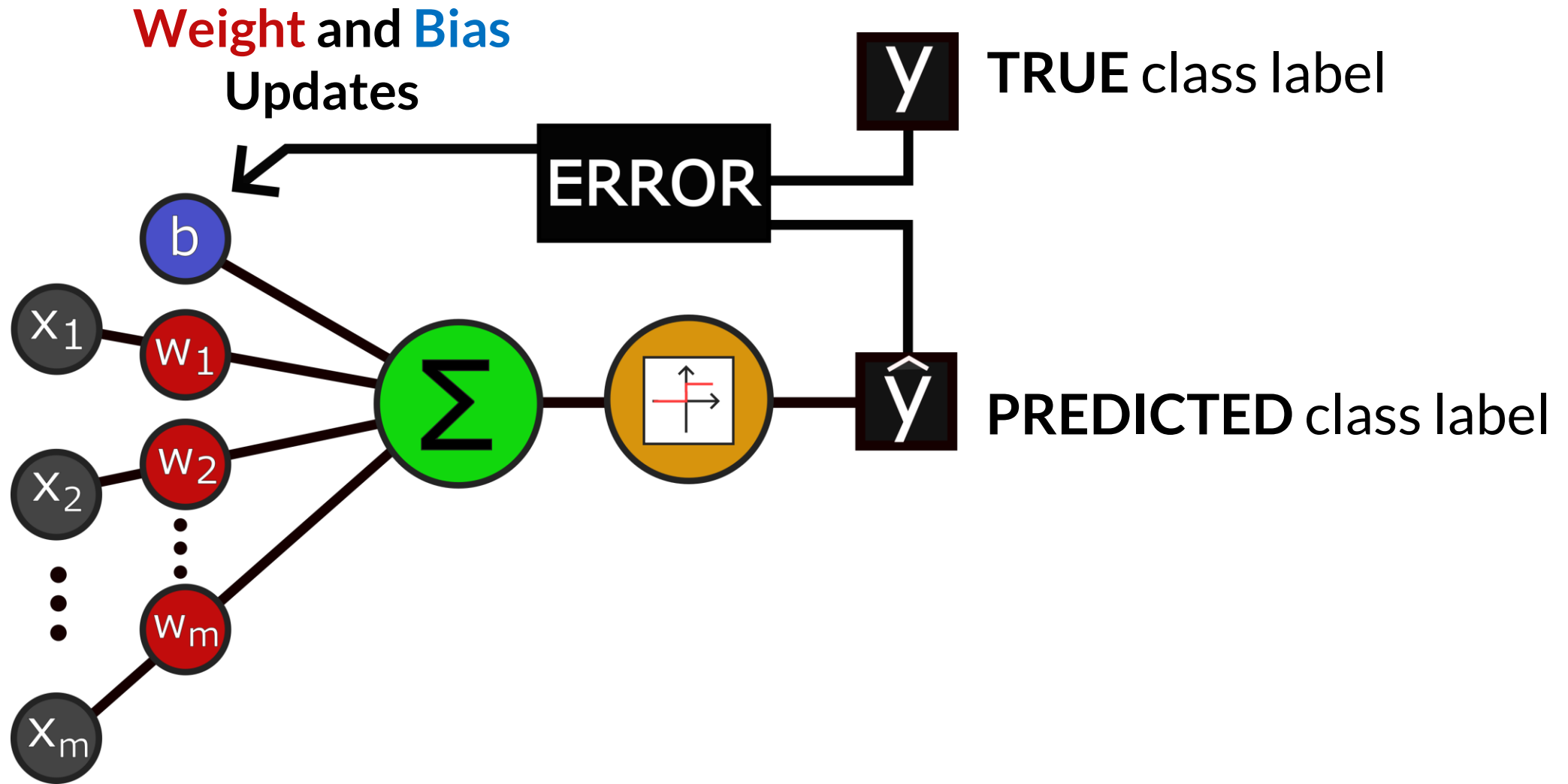
$$\begin{aligned} x_i w_i + \dots + x_n w_n - \theta &= 0 \\ x_i w_i + \dots + x_n w_n + \mathbf{b} &= 0 \end{aligned}$$

The term  $b$  is known as the **bias** – using the bias makes the threshold value trainable along with the weights



# Training a Linear Neuron - Overview

11



**Training is the process of updating the *weights* and *bias* to improve the performance of the model. Training is performed iteratively**



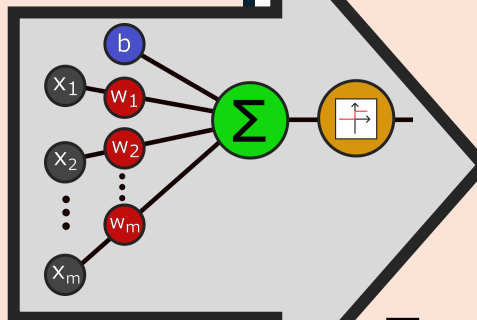
# Training - Verbal

## Training Data

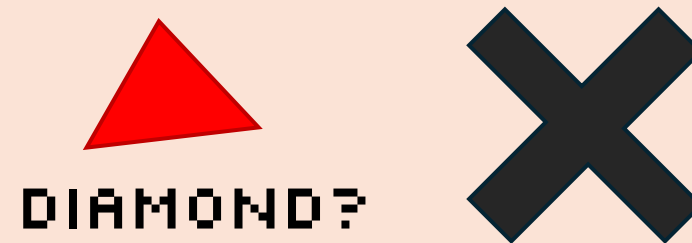


Training data consists of:

- Vectors of inputs  $\rightarrow x_i$
- The “ground truth” (i.e. correct) output for each set of inputs,  $y^{(i)}$



## Updating



For all training data calculate **model prediction**,  $\hat{y}^{(i)}$ , then:

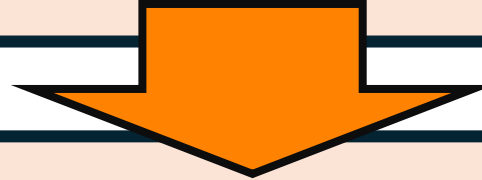
- If the prediction is correct, do nothing
- If the prediction is wrong, update the weights and bias to reduce error

Repeat until the bias and weights stop changing

# Training Algorithm - Math

13

1) Initialize the bias and weights either to zero or to small random numbers



2) For each example in the training set:

- Compute the output using the current bias and weights:  $\hat{y}^{(i)} = w^T x^{(i)} + b$
- Update the weights and the bias:
  - $w_j = w_j + \Delta w$  where  $\Delta w = \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)}$
  - $b = b + \Delta b$  where  $\Delta b = \eta(y^{(i)} - \hat{y}^{(i)})$



3) Repeat until prediction matches output for all examples in training set



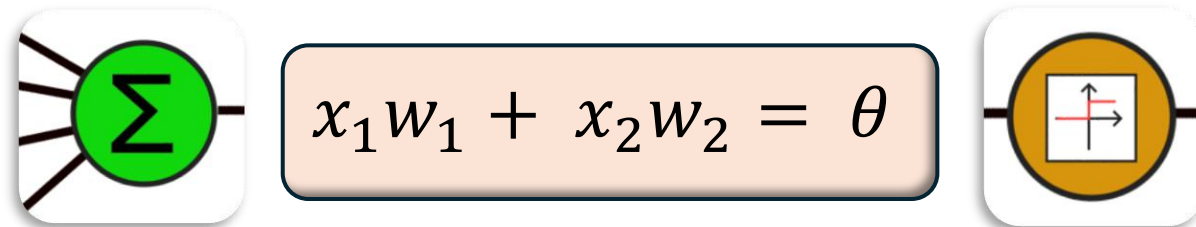
# Training: Vocabulary Review

- **Epoch:** One complete pass through the training dataset
- **Batch:** For large training datasets the model usually processes smaller subsets known as “batches” and updates parameters after each batch
- **Learning Rate:** Controls the magnitude of weight updates - a larger learning rate means the weights change faster.



# The Problem of Separability

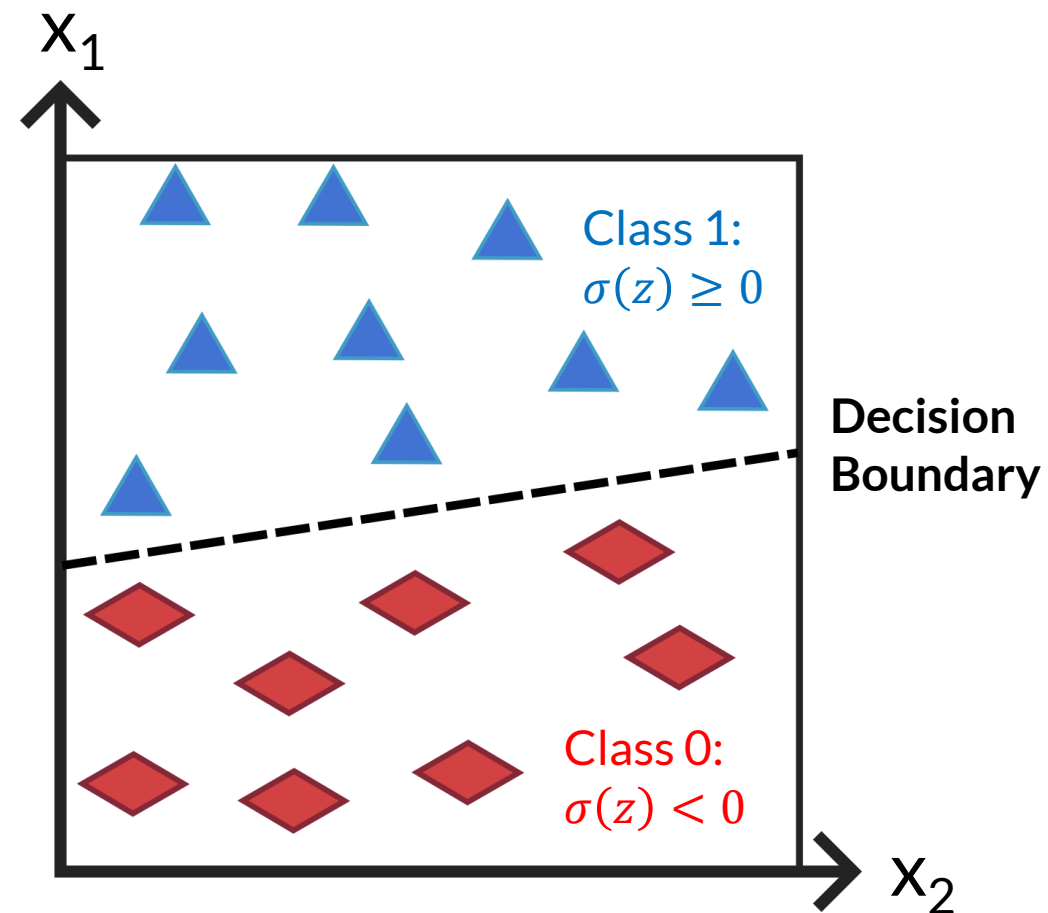
Consider a linear neuron that takes two inputs,  $x_1$  and  $x_2$ :



Rearrange:

$$\begin{aligned} x_2 w_2 &= \theta - x_1 w_1 \\ x_2 &= ax_1 + b \end{aligned}$$

Results in linear equation – **decision line/plane**

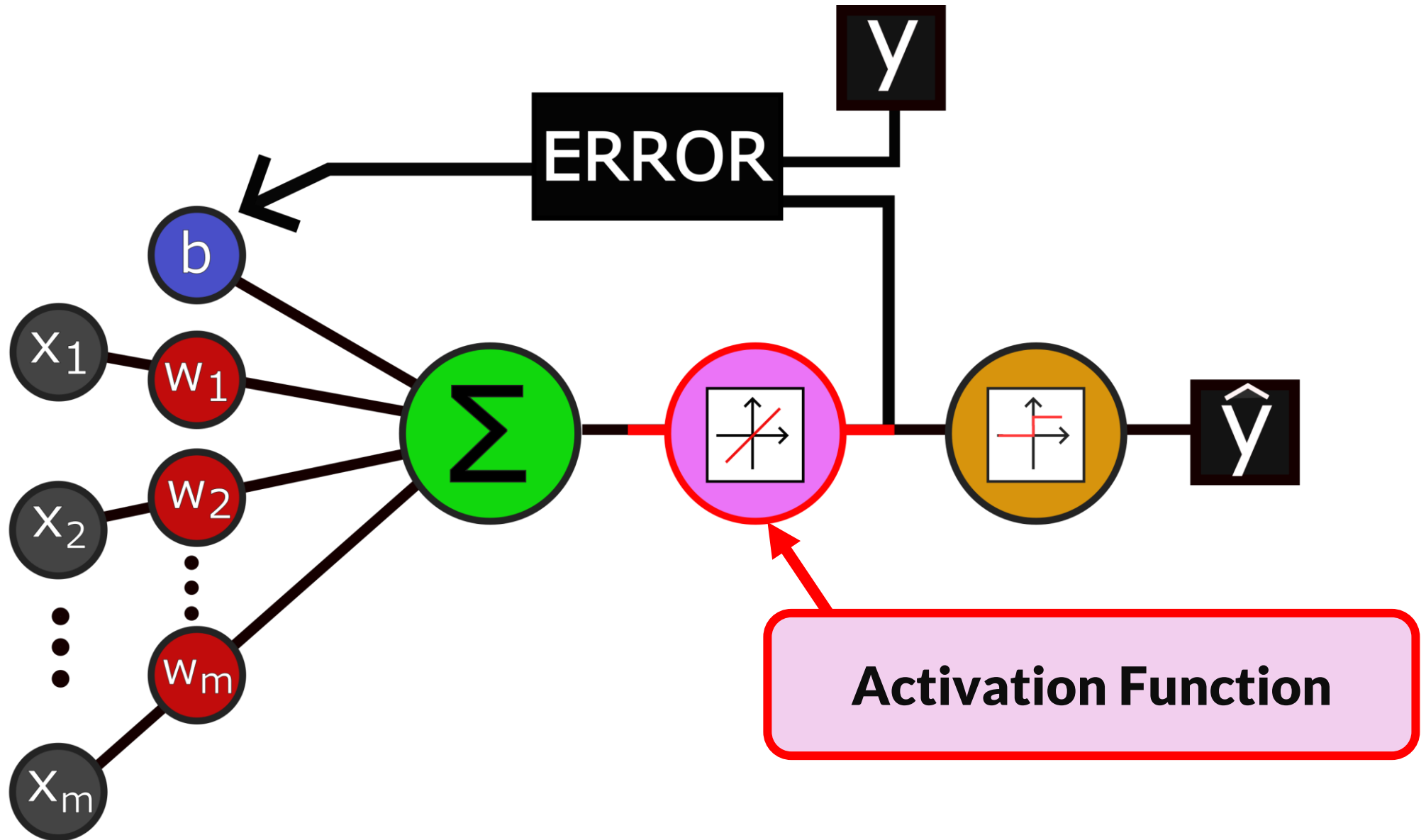


Single neurons can only solve problems that are ***linearly separable***



# Adaptive Linear Neuron (Adaline)

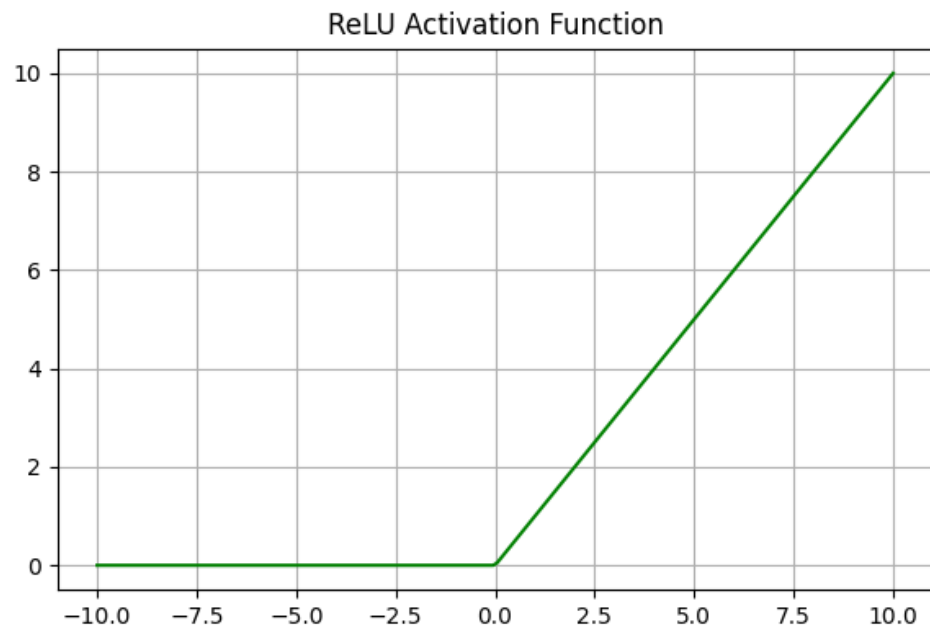
16





# Activation Functions:

The activation function takes the input and determines the output:



*If Input  $\leq 0$ :*

***Output = 0***

*If Input  $> 0$ :*

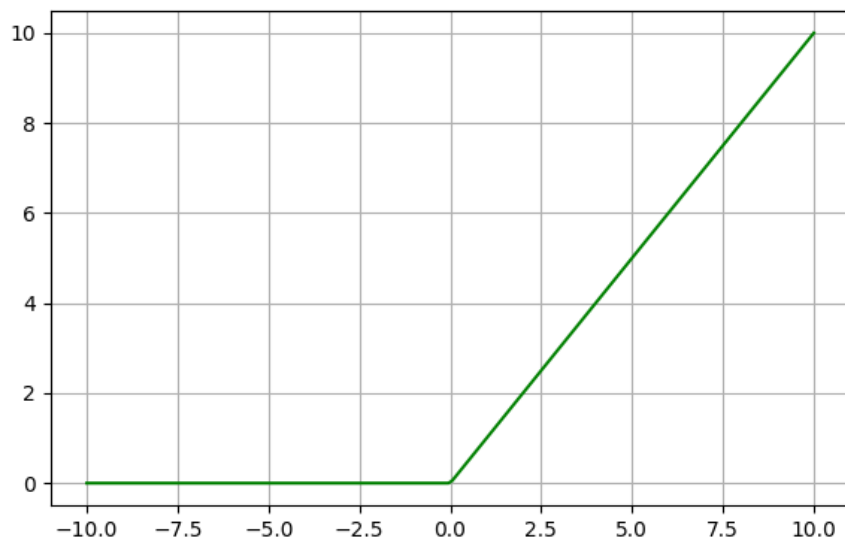
***Output = Input***

- Activation functions introduce **non-linearity** to neural networks
  - otherwise even deep networks can only solve linear problems
- Decision surface can be curved – **can learn complex patterns**



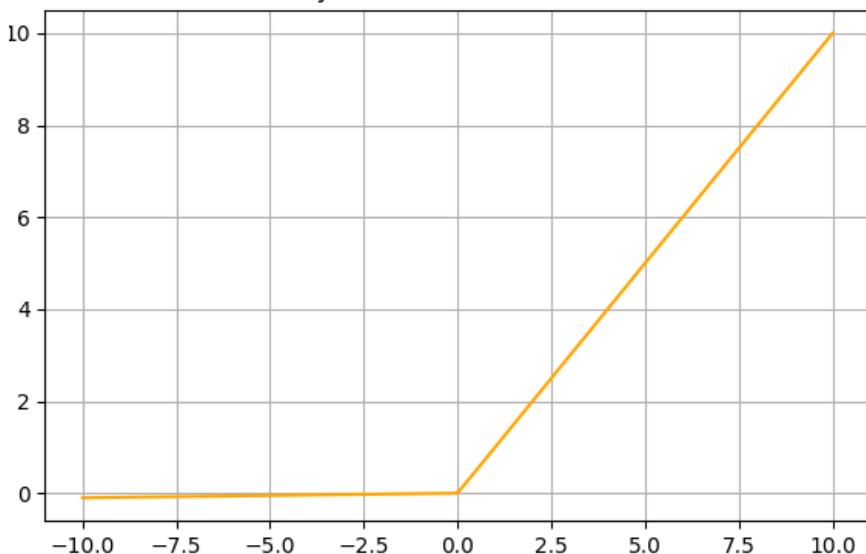
# Choosing Activation Functions

ReLU Activation Function



- **Rectified Linear Unit:** If input is negative, output is zero. If input is positive, output = input
- Often used for image data or in other cases where negative inputs are not physically meaningful

Leaky ReLU Activation Function

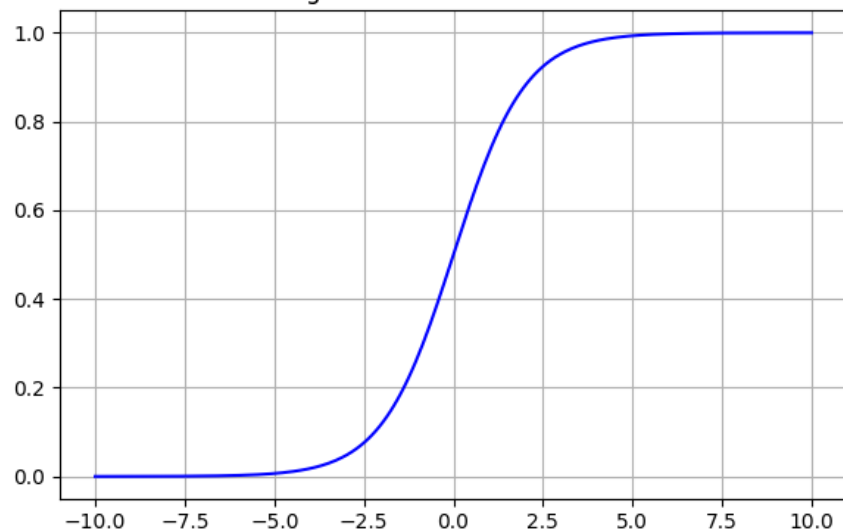


- **Leaky ReLU:** Introduce a small non-zero slope for negative inputs
- Helps avoid “dying ReLU” where negative inputs cause a node in a network to become inactive



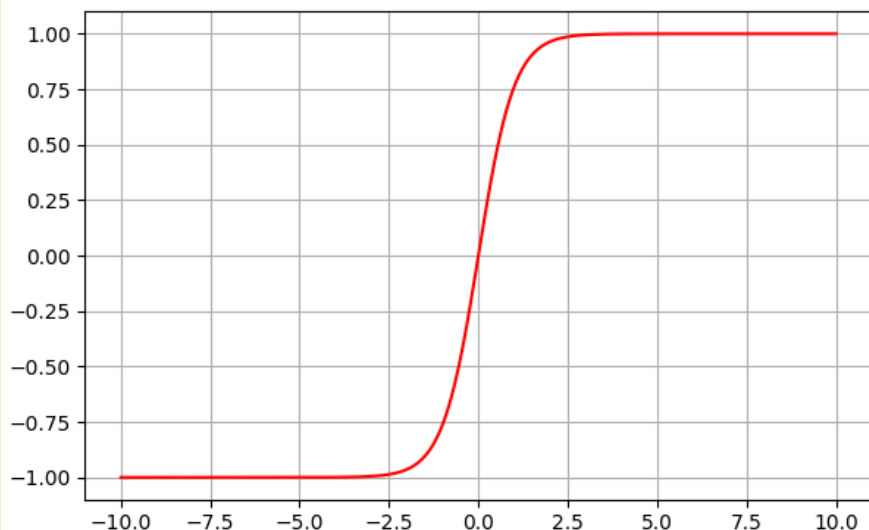
# Choosing Activation Functions

Sigmoid Activation Function



- **Sigmoid:** Maps output to values between 0 and 1
- Ideal for situations where output should be interpreted as a probability

Tanh Activation Function

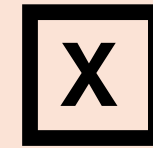
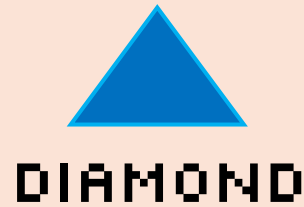
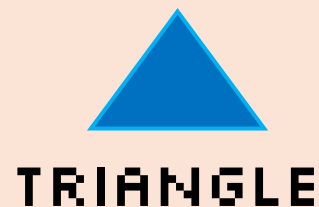


- **Tanh:** Maps output to values between -1 and 1
- Often used in hidden layers



# Error for an Adaline

- **Binary Linear Neuron:** Can only output one of two options – answer is right or wrong:



- **Adaline:** The linear activation function is used to calculate error and learn weights – the activation is continuous, so error is as well



- The error is the distance between the true value (star) and the network prediction (arrows). **We want error to be as small as possible**

# Training an Adaline – Loss Function

21

- We train an Adaline by minimizing a **loss function**:

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n \left( y^{(i)} - \sigma(z^{(i)}) \right)^2$$

**Model Prediction** (arrow pointing to  $\sigma(z^{(i)})$ )

**Ground Truth** (arrow pointing to  $y^{(i)}$ )

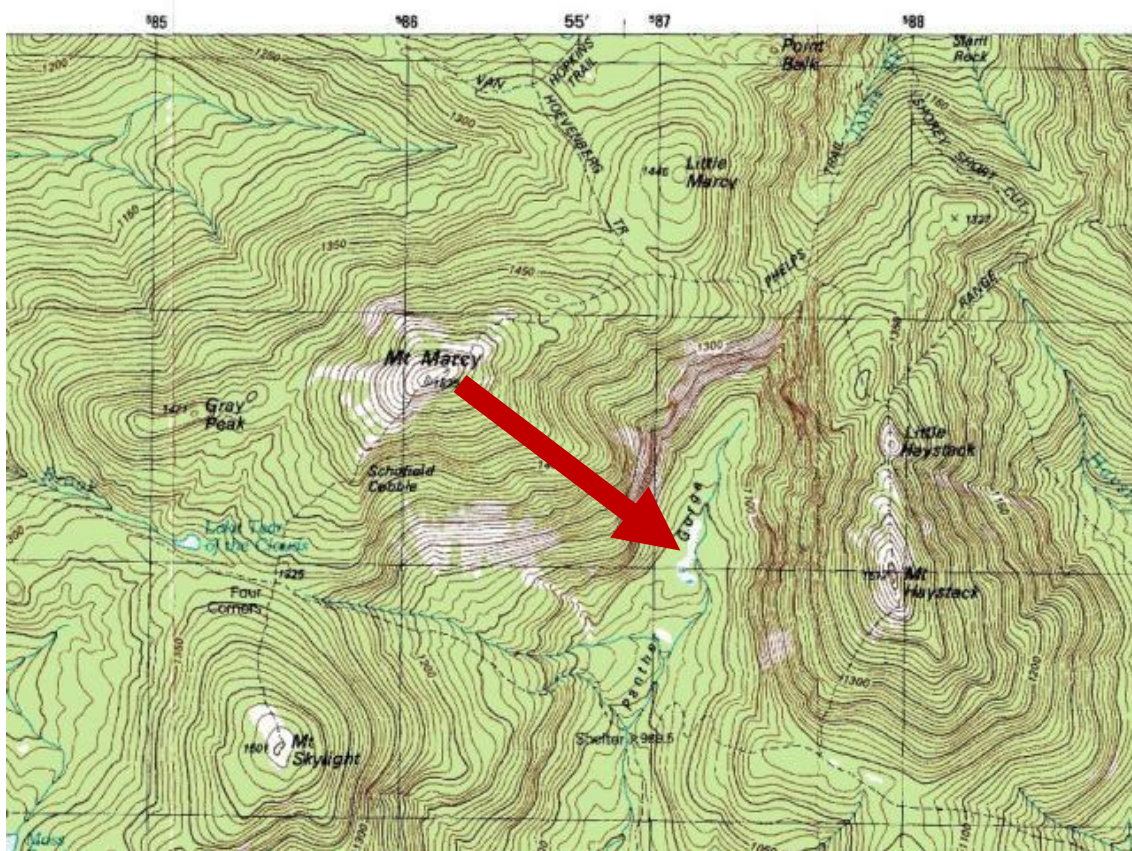
- A loss function quantifies the difference between the network prediction and the ground truth
- The above is called the Mean Squared Error (MSE) loss and is common for regression problems

**Loss functions quantify error for Adaline; training occurs by minimizing the loss**



# Gradient Descent - Analogy

- For every combination of weights and bias there is a corresponding value of the loss. This forms a loss surface/hypersurface. To train the network we must move to the minimum of the surface



- You are on a hill, and want to reach the valley (minimum) nearby
- To reach the minimum, make a series of steps, always making sure to **move down hill**





# Gradient Descent - Math

- To move “downhill” we need the slope of the loss function
  - To find the slope of a function, take the derivative (in this case **partial derivatives** since loss is a multivariate function)

1) Start at an arbitrary position on the loss surface.

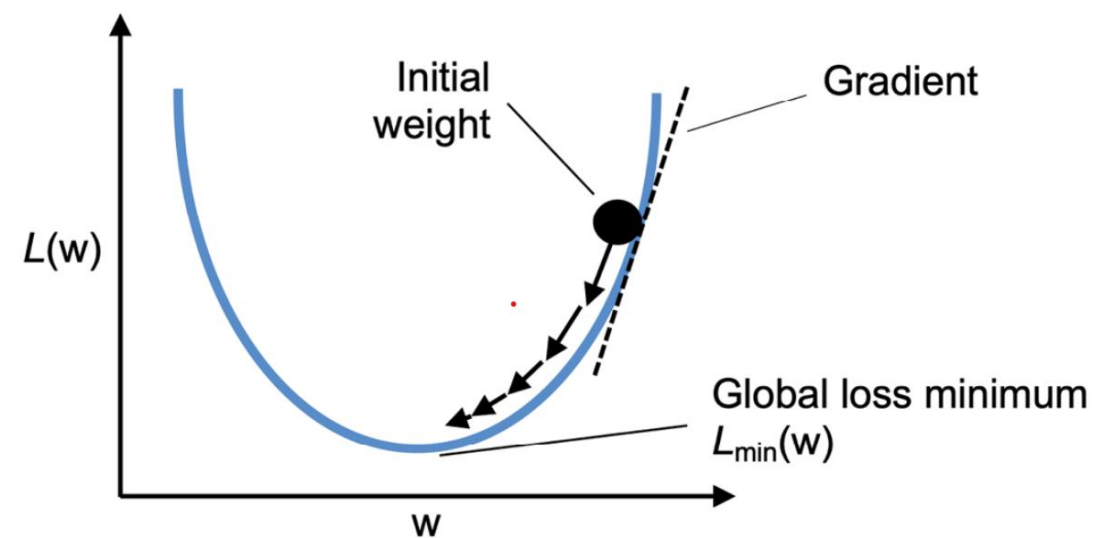
2) Use the learning rule:

$$\Delta w = -\eta \nabla_w L(w, b)$$

$$\Delta b = -\eta \nabla_b L(w, b)$$

$$w := w + \Delta w$$

$$b := w + \Delta b$$



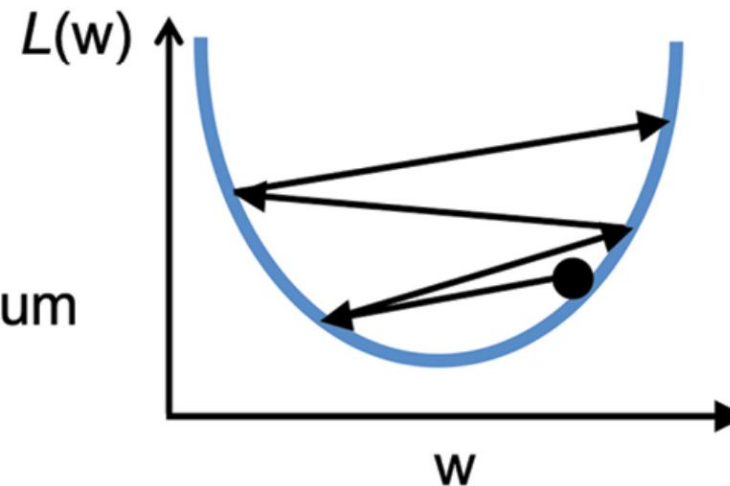
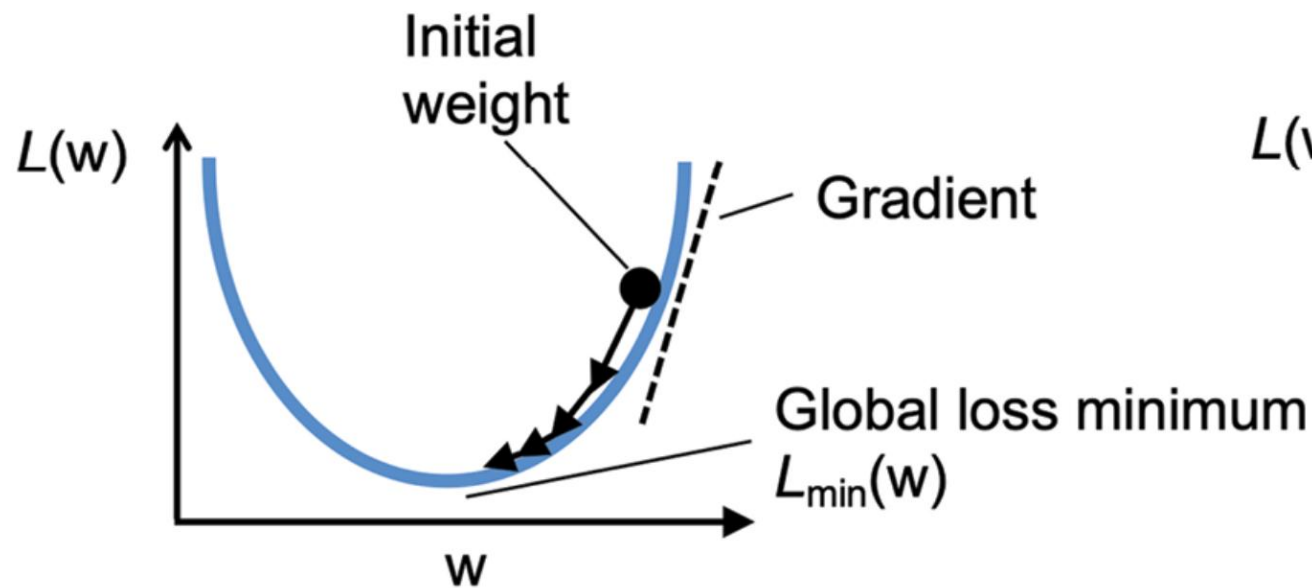
$\eta$  = Learning Rate

**Determines how large each “step” is**



# Role of Learning Rate

- Learning rate,  $\eta$ , determines how quickly weights and bias change – how large each “step” is. Too small and the network will take a very long time to reach the minimum. Too large and it may overshoot the minimum and fail to converge.



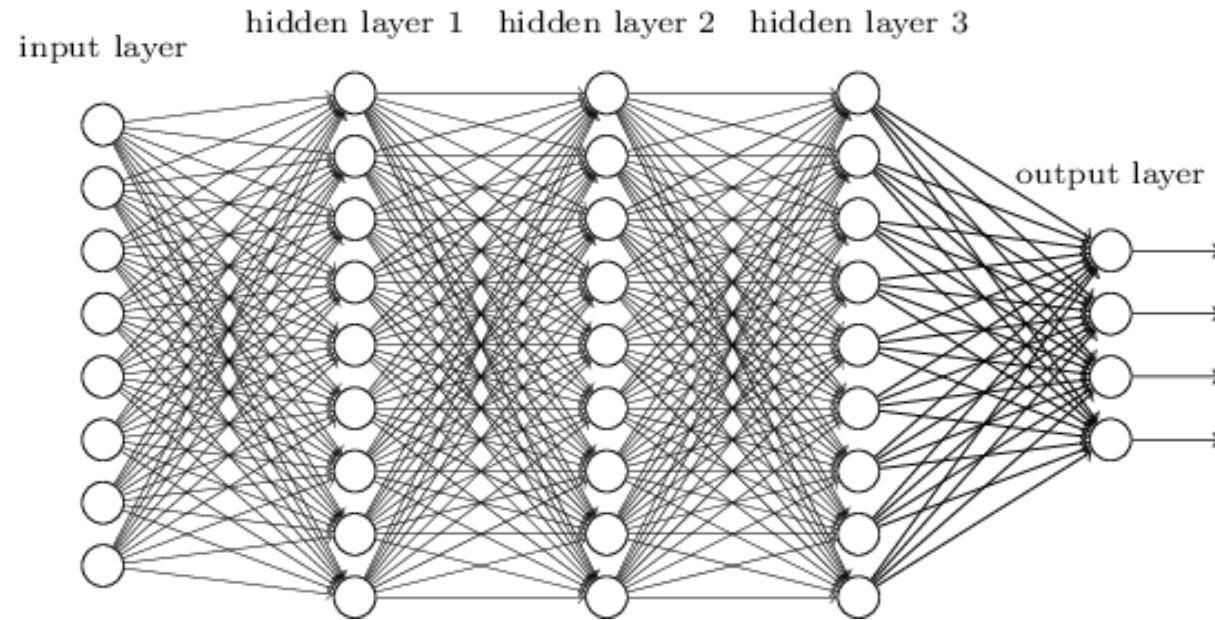


# II. FROM NEURONS TO NETWORK

# Putting Neurons Together

26

- Deep architectures allow networks to learn complex patterns in data (so long as they have non-linear activation functions) – how does a multi-layer network work?



- Composed of multiple layers of artificial neurons – the inner layers are known as “hidden layers”
  - The number of nodes per layer and the connectivity between nodes varies between networks
- Each node receives inputs, applies weights and biases, and passes outputs onto the next layer



# Training a Multilayer Network

- To train a multi-layer network, each neuron's weights and bias must be updated. To train multi-layer networks an approach known as **backpropagation** is used.
- Backpropagation consists of a forward and backward pass:
  - **Forward pass:** put training data into network at input layer, and propagate forward through the network, producing an output.
  - Calculate the loss based on the network prediction and the ground truth
  - **Backward pass:** Compute the gradient of the loss function with respect to each neuron's weights by chain rule
  - Update weights

***Multi-layer networks are trained by backpropagation, which moves layer by layer to determine appropriate updates to the network weights***



# Metrics

***Once a network is trained, the performance can be evaluated according to several possible metrics, including:***

- **Accuracy:** Defined as the ratio of correctly predicted observations to the total observations.
- **Precision:** The ratio of correctly predicted positive observations to the total predicted positive observations. Important in scenarios where the cost of a false positive is high.
- **Recall (Sensitivity):** The ratio of correctly predicted positive observations to all observations in the actual class. Crucial in situations where missing a positive instance is costly.



# There are Many Types of Neural Networks

- Multilayer perceptron
- Convolutional Neural Networks
- Autoencoders
- Physics informed neural networks
- Etc.



# Where to Start:

## Problem-Centric Approach:

- Always start with the problem at hand: Analyze the nature of input data and desired output
- Choose a network architecture that aligns with the type and structure of your data
- Select a loss function that reflects the objective of the problem
- Metrics should be chosen based on what measures success for your specific task



# III. CONVOLUTIONAL NEURAL NETWORKS

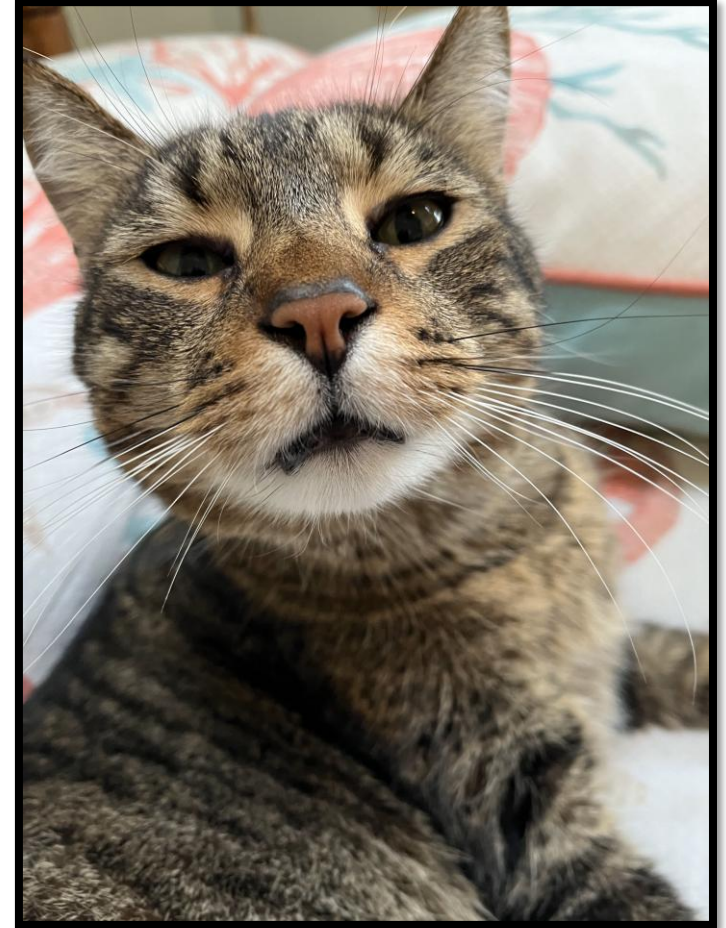


# One Type of Problem: Image Classifier



**DOG**

**VS.**



**CAT**



# The Tool for the Job: CNN

33

- Convolutional neural networks (CNNs) are specialized networks designed for processing image data (can also be used to audio or other signals)
- CNNs can process data spatially, identifying features in different parts of the image
- How do we represent an image as data?



# Representing an Image Numerically

34



3742 \* 4624 Pixels

- An image is represented numerically as an array of values. RGB images contain 3 channels for red, green and blue

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |   |   |
| 0 | 1 | 0 | 0 | 0 | 0 | 1 |   |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 1 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |
|   | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
|   |   | 0 | 0 | 1 | 0 | 1 | 0 |

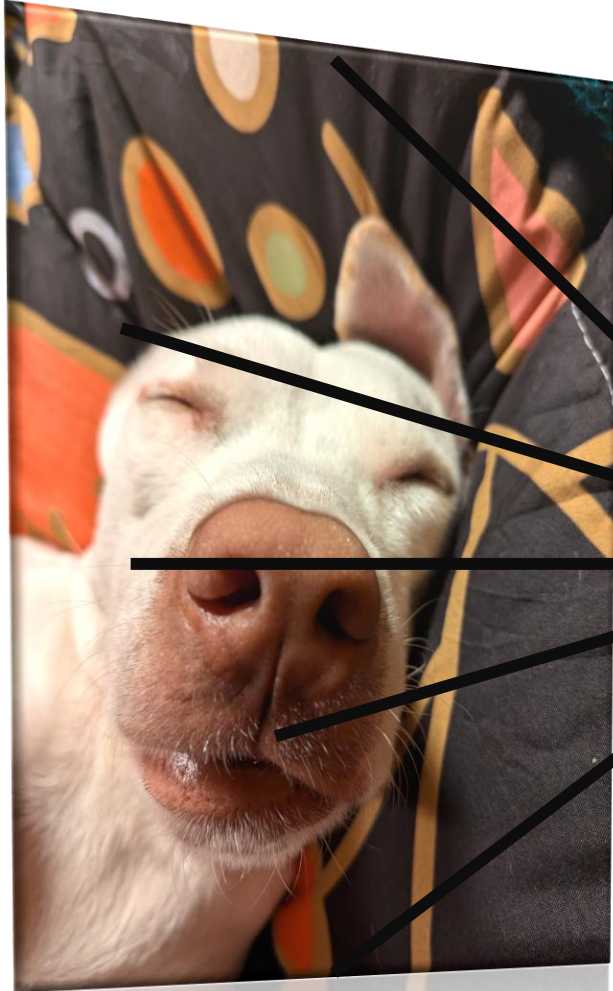
$$3743 * 4624 * 3 = 51,909,092$$

**This picture is really big!**



# Image as Input to Neural Network

35



If a fully connected network is used to process this image and every neuron has a weight for every pixel value, then every neuron needs:

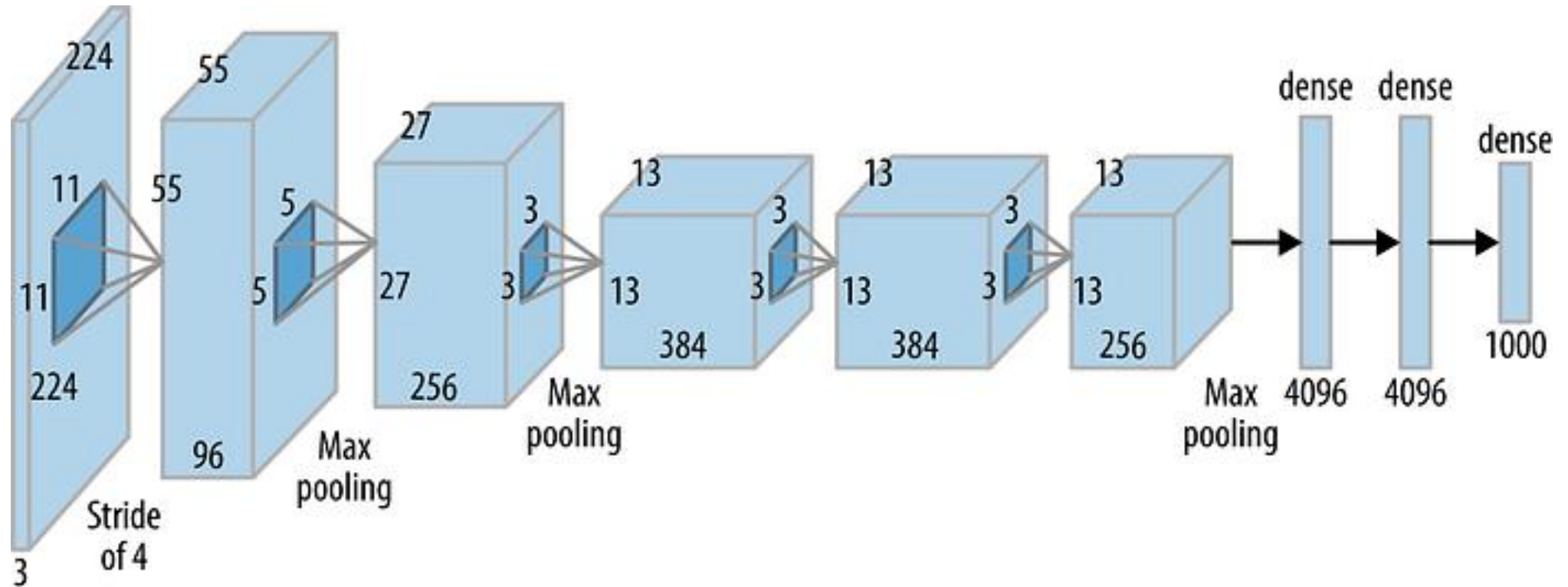
- 51,909,024 weights for RGB
- 17,303,008 weights for single channel grayscale

Even for more reasonably sized input images (say 128 by 128) it is highly inefficient to process data this large with a standard fully connected network!

**Convolutional Neural Networks are much more parameter efficient for processing images than dense networks. Even so, DCNN architectures often employ downsampling/pooling to decrease the number of parameters.**



# DCNN Structure



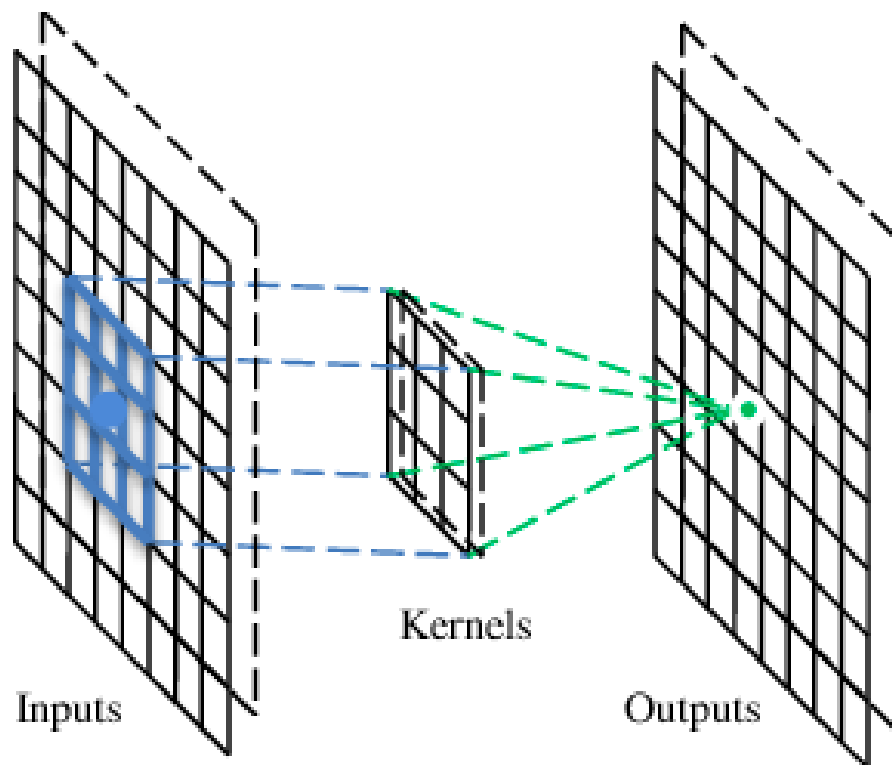
- Convolutional Neural Networks contain 3 types of layers:
  - **Convolutional layers**
  - **Pooling**
  - **Fully Connected/Dense layers**



# CONVOLUTION

# Convolutional Layers

- What is a convolutional layer?
  - A convolutional layer has a number of **filters (AKA Kernels)**
  - Filters detect specific patterns in input data. Filters can be used for feature extraction from images



- With kernels, each neuron only “listens” to a small portion of the input images
- This means the neurons don’t need as many weights



# How do Filters Work:

- Convolution operates on two images (remember, an image is stored as a matrix of pixel values) - one input is the image, and the second acts as a filter
- Kernels, which are smaller than the total size image, are slid across the image, analyzing one region at a time
  - The image matrix is multiplied by the filter matrix to form an output, the filter matrix is then advanced across the image
  - The **stride** determines how many pixels the filter region moves by – bigger stride = less overlap
- Different kernels accomplish different tasks:
  - Sharpen
  - Blur
  - Edge detection



# Convolution - Visual

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |

6 x 6 Image

The network must learn these kernels

|    |    |    |
|----|----|----|
| 1  | -1 | -1 |
| -1 | 1  | -1 |
| -1 | -1 | 1  |

Filter 1

|    |   |    |
|----|---|----|
| -1 | 1 | -1 |
| -1 | 1 | -1 |
| -1 | 1 | -1 |

Filter 2

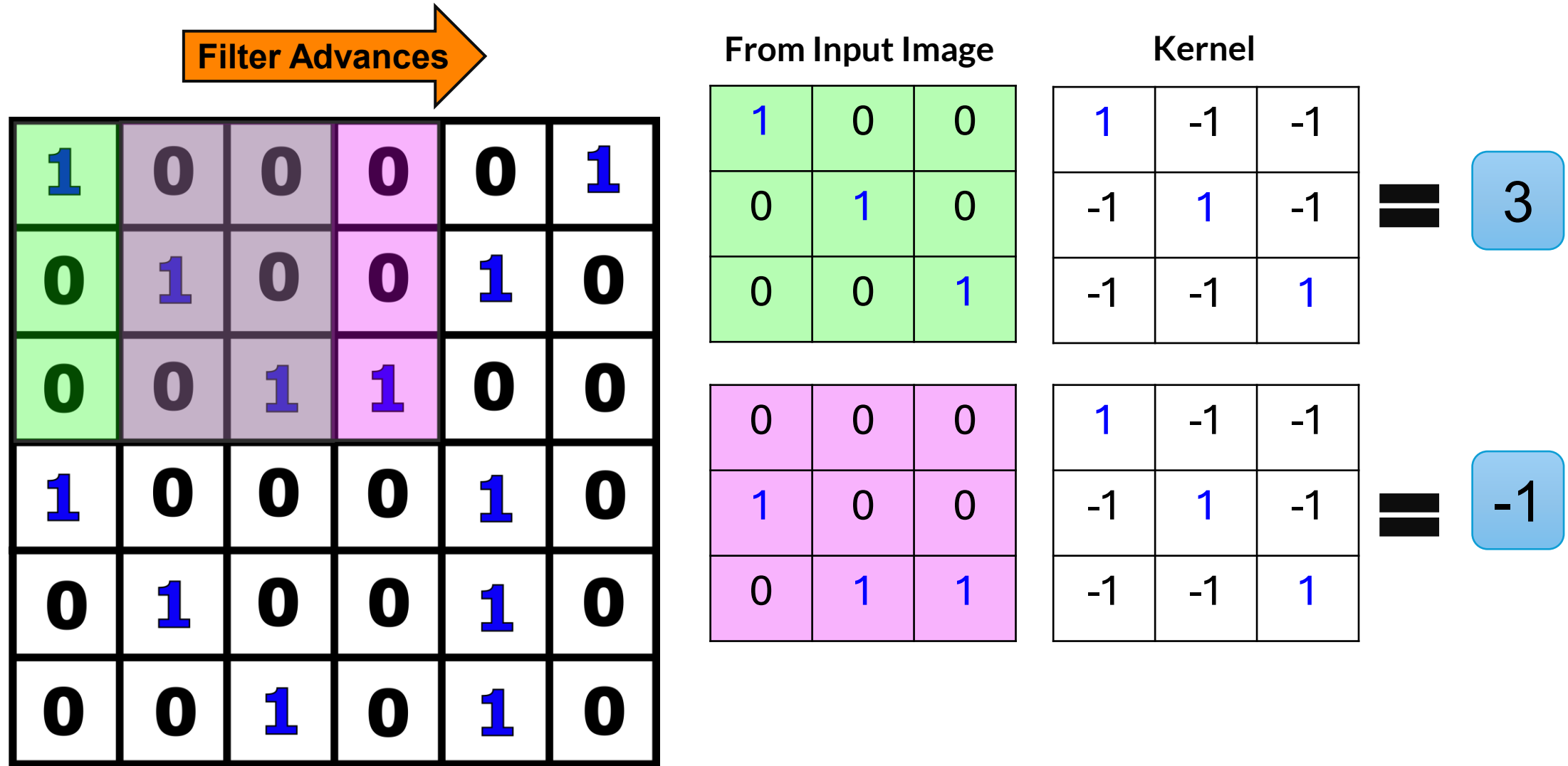
⋮ ⋮

Each filter detects a small pattern (3 x 3).





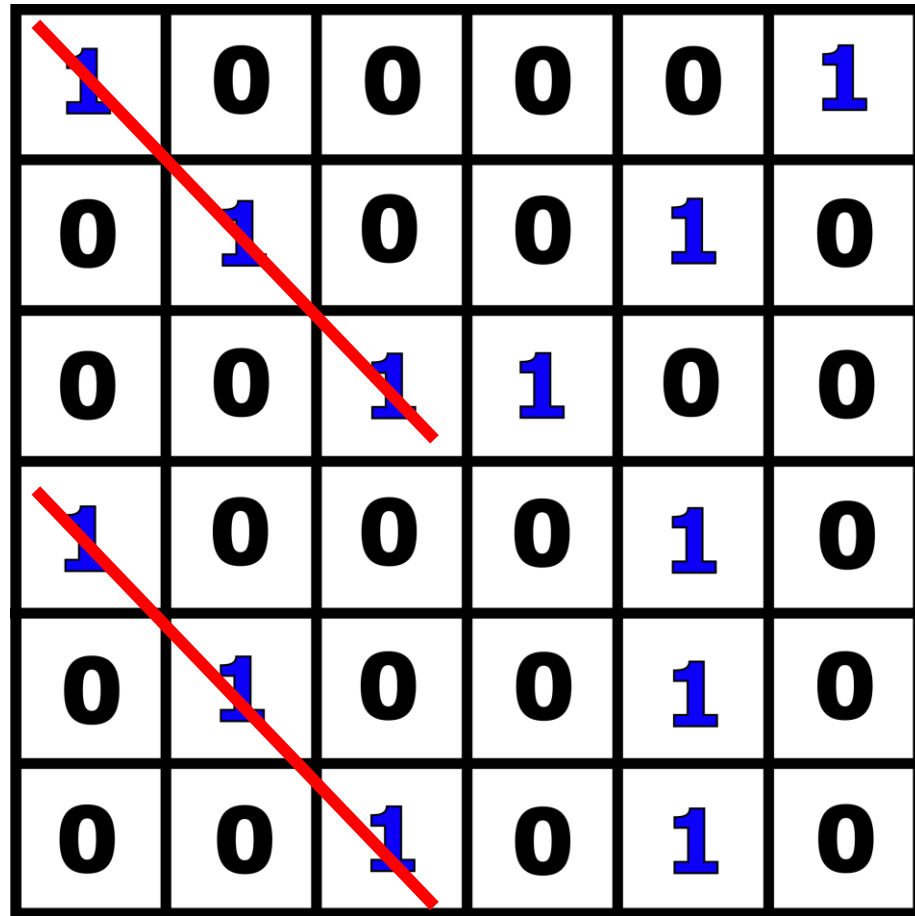
# Convolution - Visual



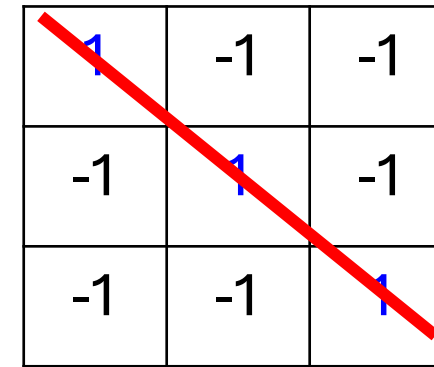
**The filter is advanced over the image, at each location the dot product is taken, resulting in a scalar value. A new matrix is built from the resulting scalars**



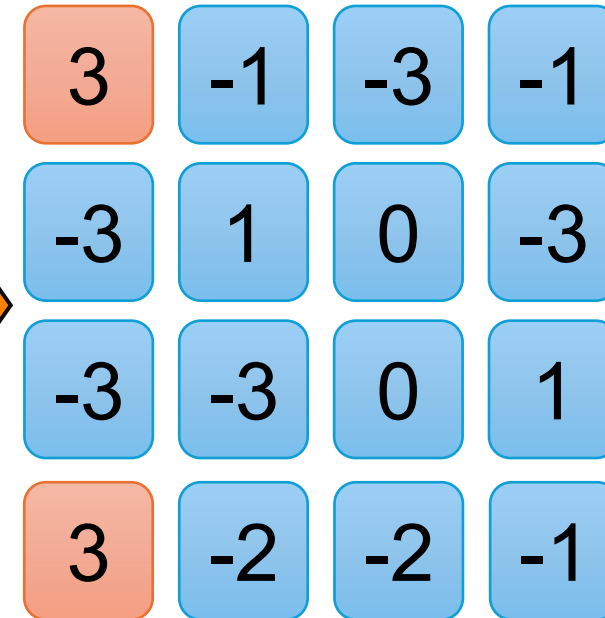
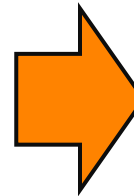
# Convolution - Visual



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |



|    |    |    |
|----|----|----|
| 1  | -1 | -1 |
| -1 | 1  | -1 |
| -1 | -1 | 1  |



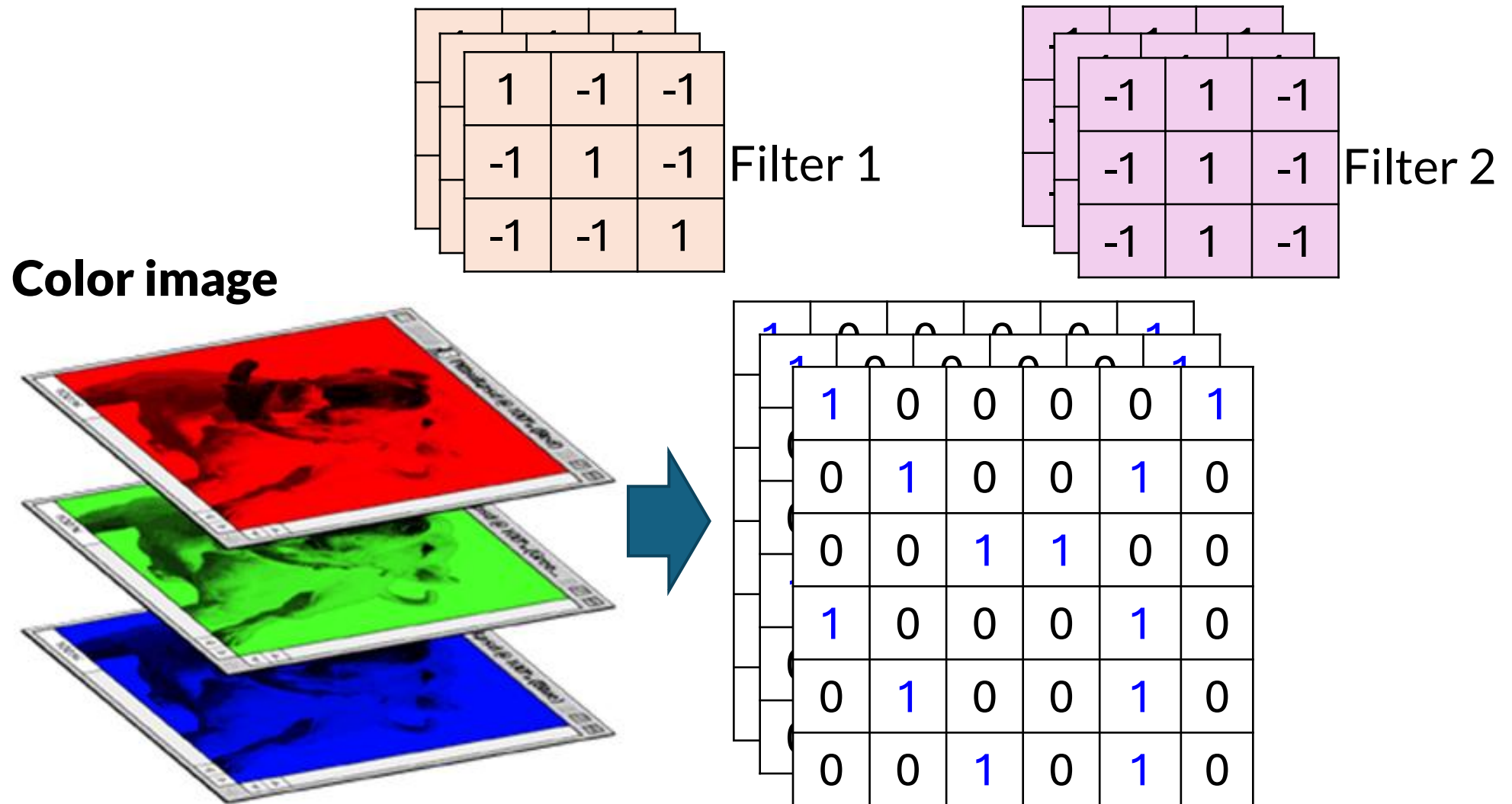
|    |    |    |    |
|----|----|----|----|
| 3  | -1 | -3 | -1 |
| -3 | 1  | 0  | -3 |
| -3 | -3 | 0  | 1  |
| 3  | -2 | -2 | -1 |

**The filter identifies specific patterns in the input image**  
**Repeat this process for all learned filters to make a features map**



# But what about color?

43



# Stride

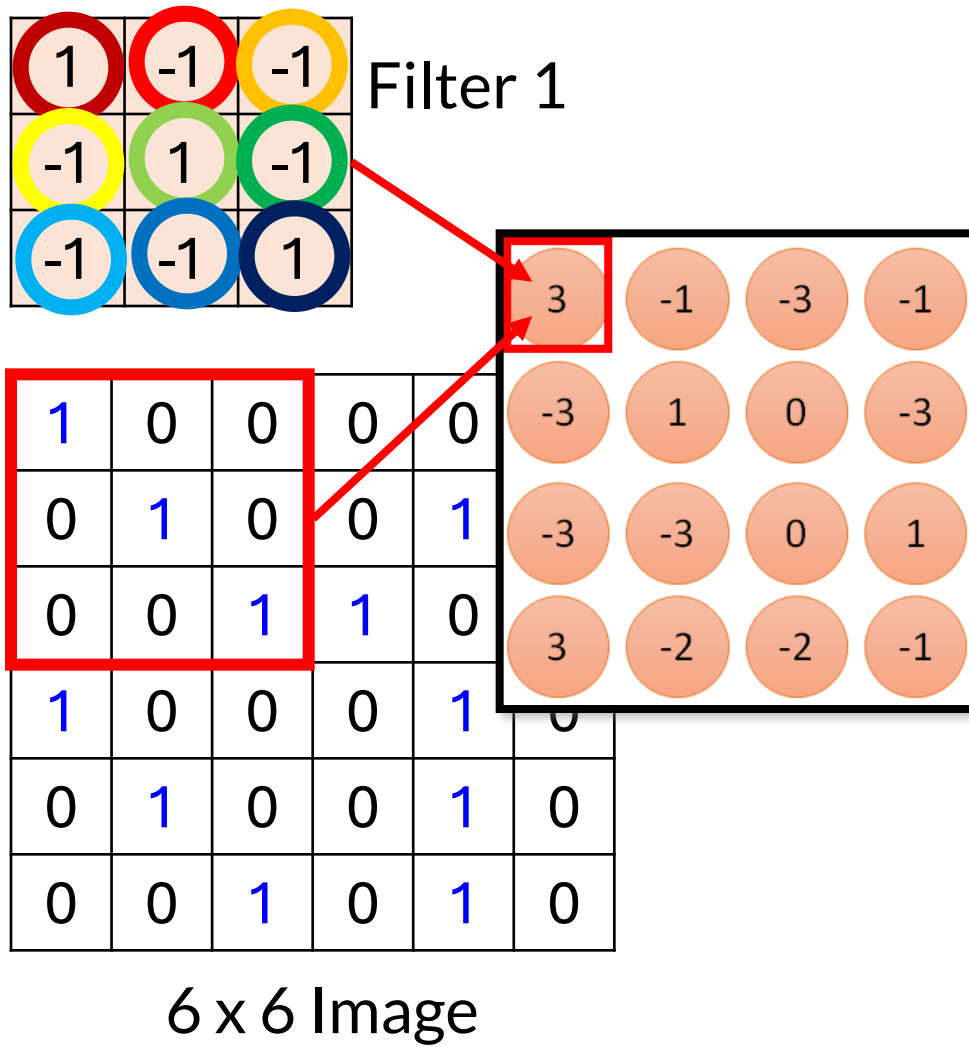
|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |

Stride = 1

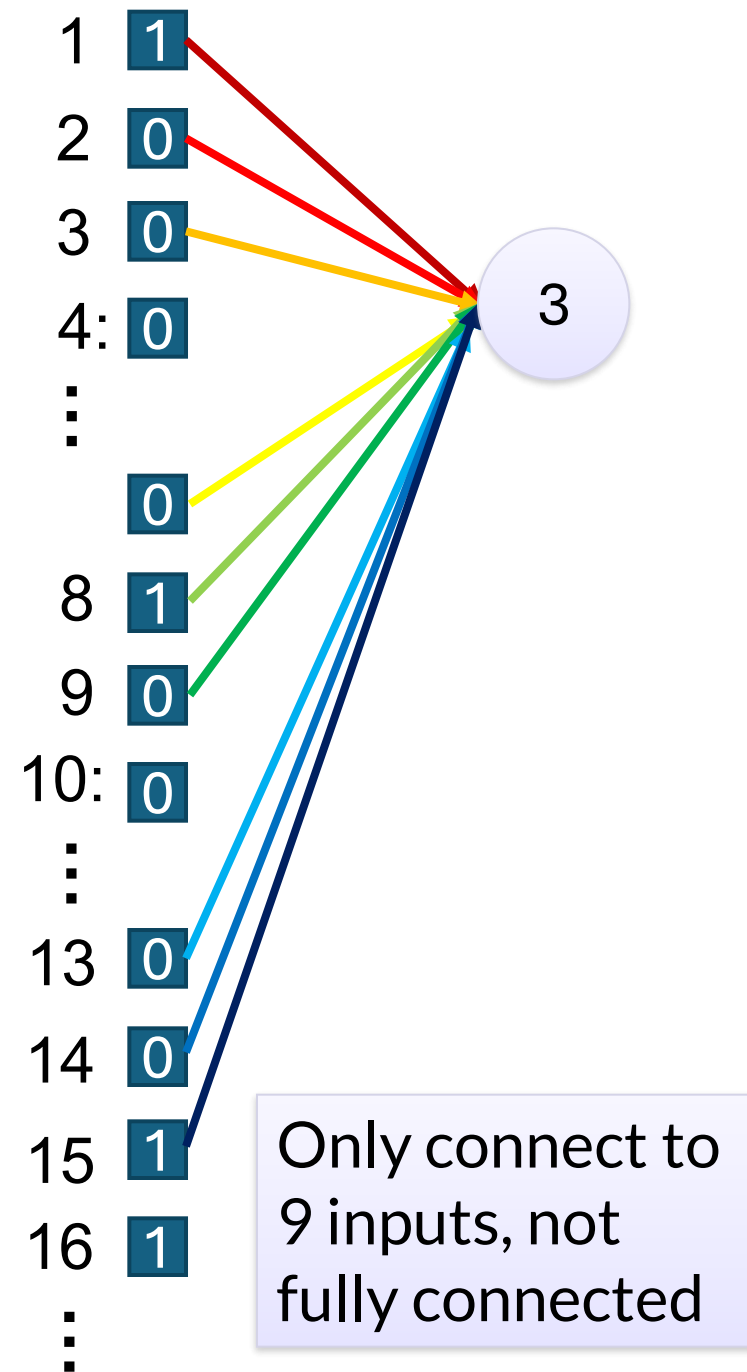
|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |

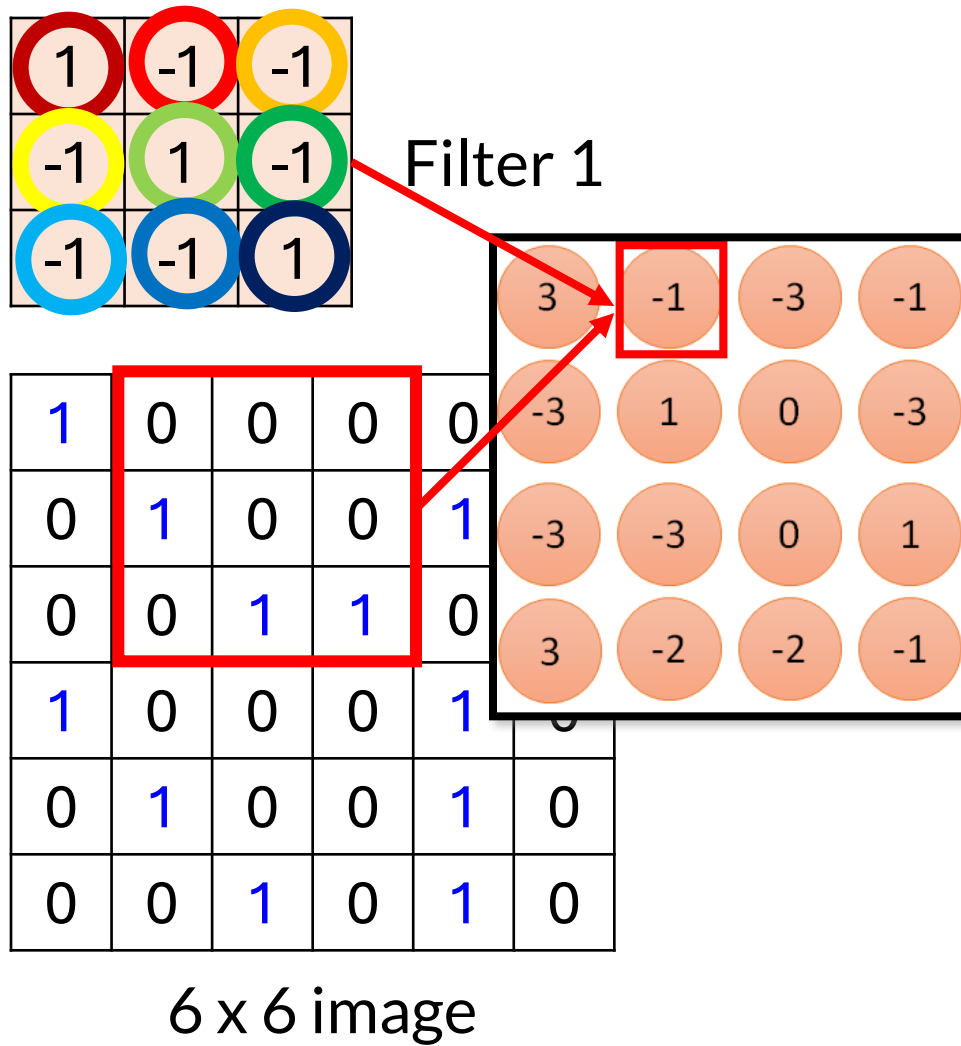
Stride = 2



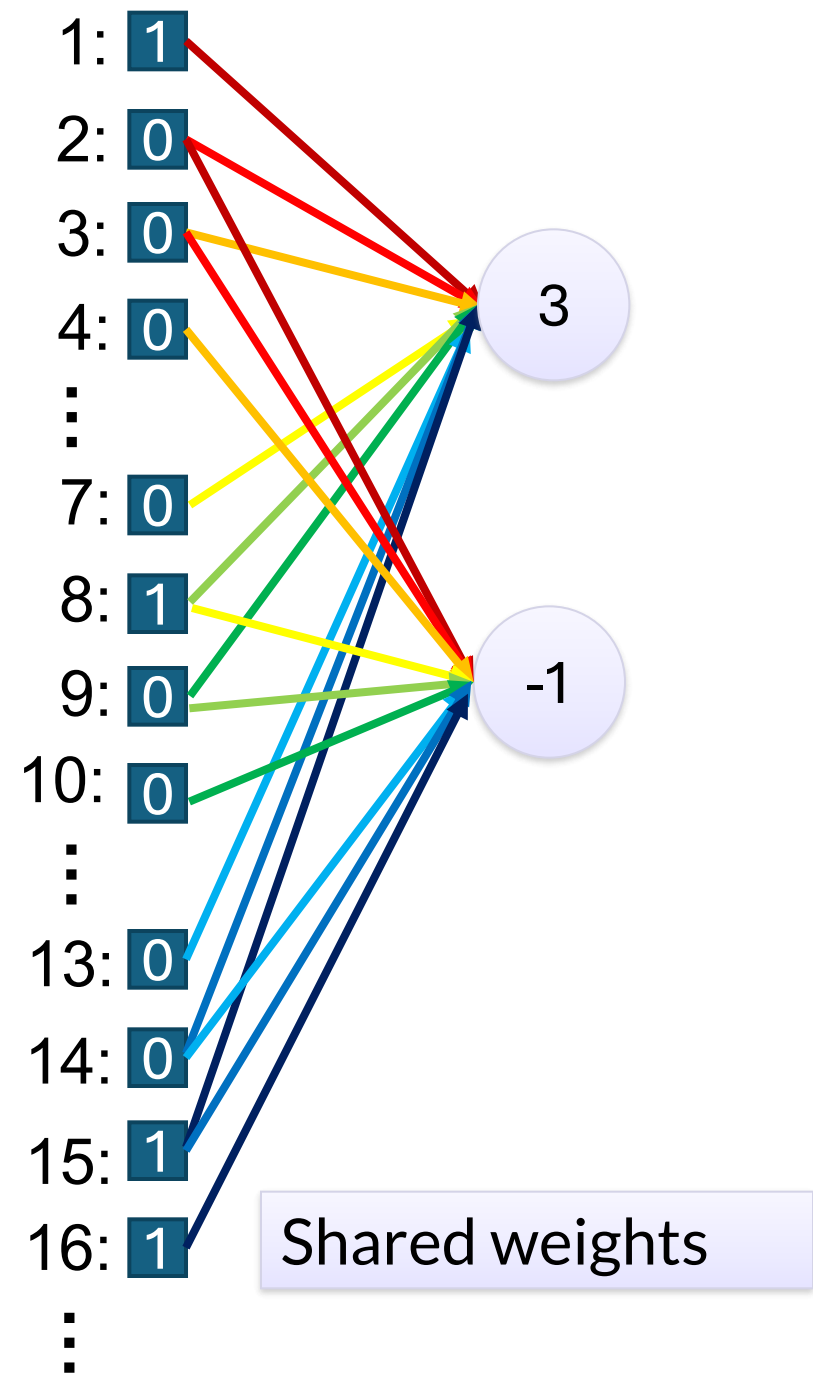


Fewer parameters!





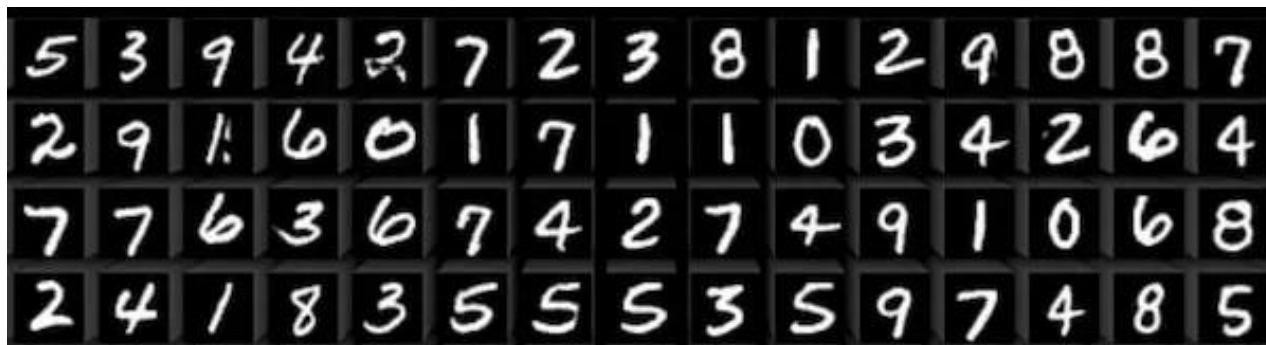
Fewer parameters



# Result of Convolution - MNIST

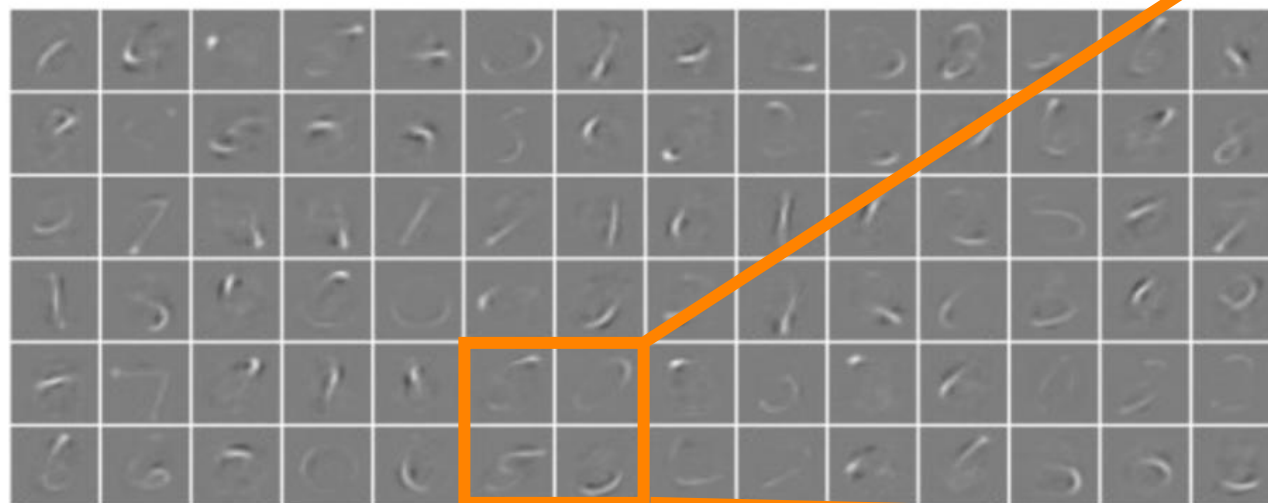
47

Example training data – handwritten digits



- A CNN trained on the MNIST database of handwritten characters produces “activations” that contain the characteristics of the individual digits

Result of First Convolution



From Nash et al. “An Introduction to Convolutional Neural Networks”



# POOLING



# Pooling Layers

49

- Pooling makes an image smaller, so that the network requires fewer parameters
- Divide the image into non-overlapping regions. Create a new pixel value from each region, resulting in a smaller output array.
  - **Max Pooling:** Output for each region is max of pixel values in region
  - **Average Pooling:** Output for each region is average of pixel values in region

**Pooling layers make the image smaller *without changing the object***



# Max Pool and Average Pool

50

|    |    |    |    |
|----|----|----|----|
| 3  | -1 | -3 | -1 |
| -3 | 1  | 0  | -3 |
| -3 | -3 | 0  | 1  |
| 3  | -2 | -2 | -1 |



|   |   |
|---|---|
| 3 | 0 |
| 3 | 1 |



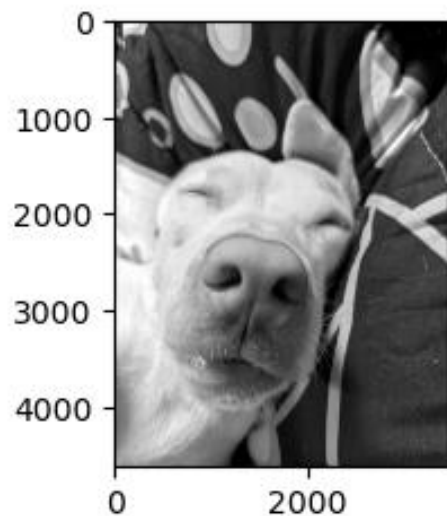
|       |       |
|-------|-------|
| 0     | -1.75 |
| -0.25 | -0.5  |



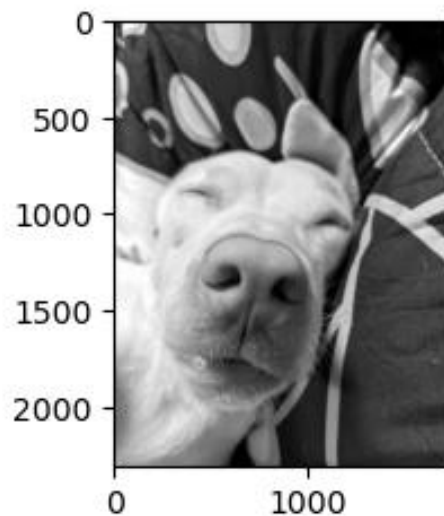
# Pooling a Real Image

51

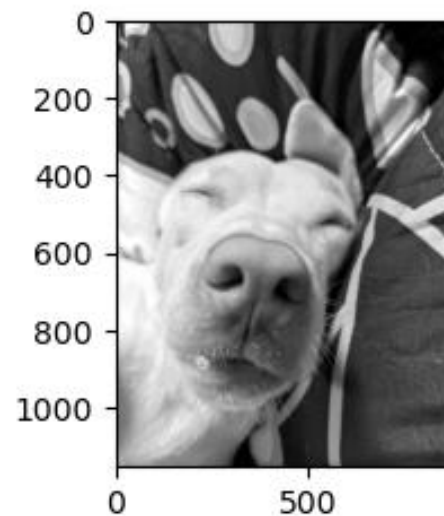
**3742\*4624**



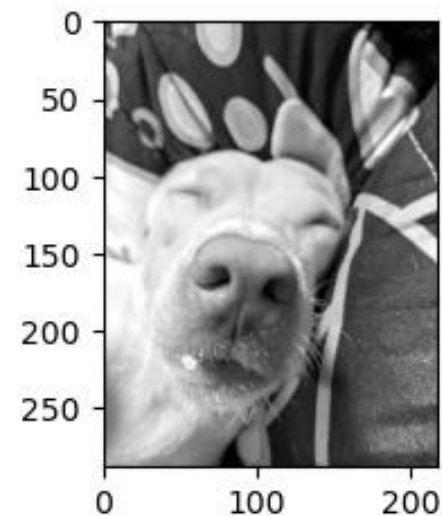
**2312\*1736**



**1156\*868**



**289\*217**



Subsampling pixels does not change the object



# ASSEMBLING THE CNN

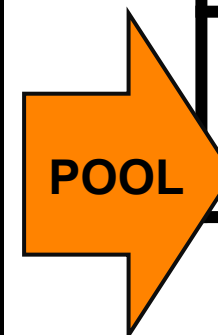
# Moving Through the Layers

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |

Input Image



|    |    |    |    |
|----|----|----|----|
| 3  | -1 | -3 | -1 |
| -3 | 1  | 0  | -3 |
| -3 | -3 | 0  | 1  |
| 3  | -2 | -2 | -1 |



|   |   |
|---|---|
|   |   |
|   |   |
| 3 | 0 |
| 3 | 1 |

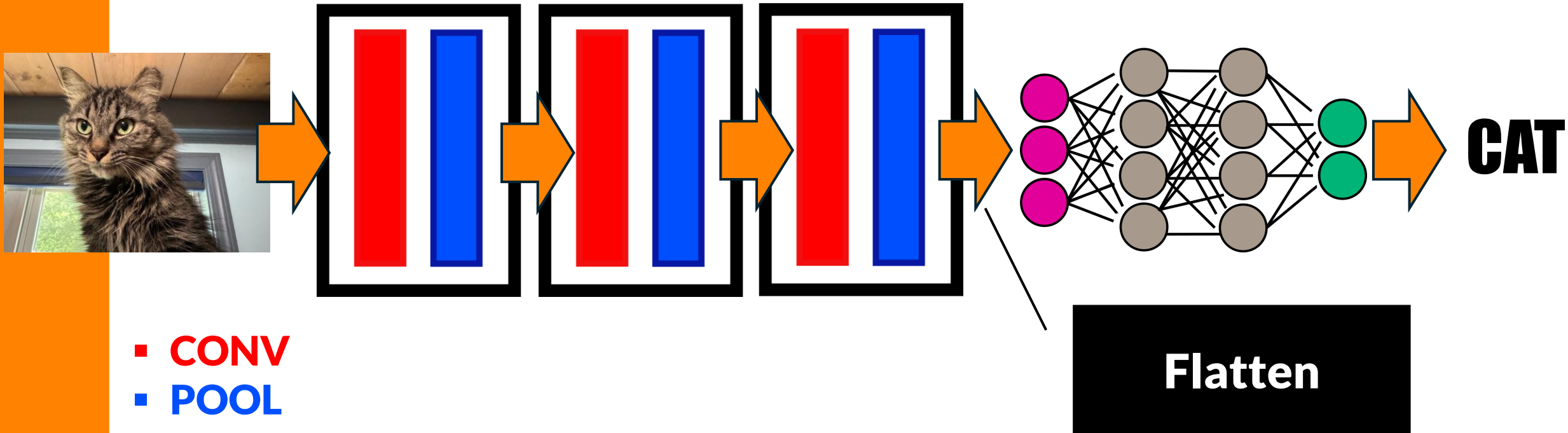
Smaller Image, 1  
channel for each  
filter

Convolving an image with a kernel generates a new image. For each convolutional layer, the number of output channels is equal to the number of filters used. This is called “*depth*”. Pooling shrinks each channel but doesn’t change depth.



# Typical Architecture

- DCNNs typically consist of alternating convolution/pooling layers:

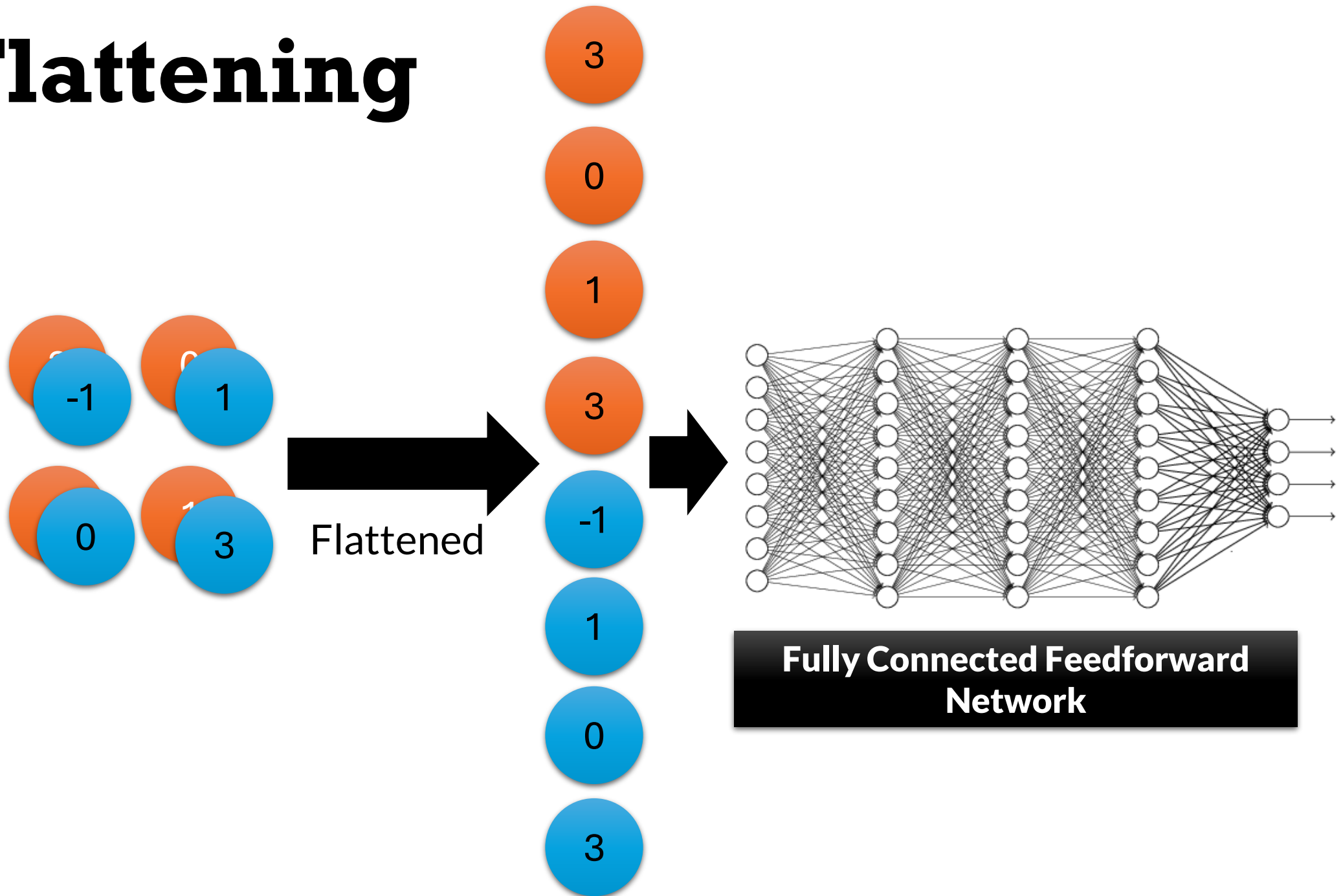


- **CONV**
- **POOL**

- After the convolutional layers, a process called “flattening” is performed and the result is fed into a dense network:
  - **Flatten Layer:** Flattens the input from a multi-dimensional tensor to a 1D tensor



# Flattening

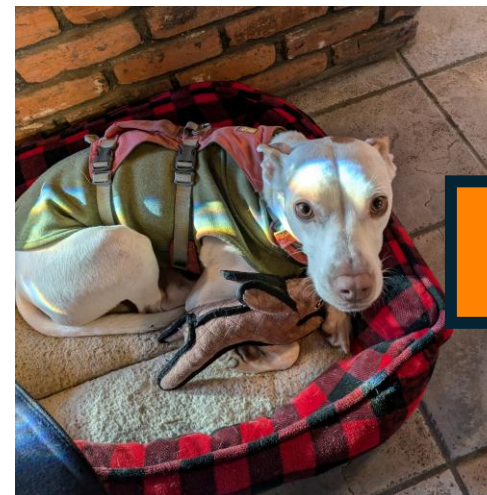




# Training

CNN's typically require **large training sets** to achieve good accuracy. If training images are limited, consider the possibility of data augmentation:

- Rotate
  - Noise
  - Crops
- 
- Typically, CNN's are trained on GPU(s) for speed



**DOG**



**STILL DOG**



**CAT**



**ALSO CAT**



# PRACTICAL IMPLEMENTATION

# Popular Frameworks (Python)



## Pros:

- Easy to learn. Models can be assembled by stacking layers
- Makes it easy to build models without having to code a lot of low-level math stuff

## Cons:

- No native GPU support on windows 😞

```
model = keras.Sequential([
    layers.Input(shape=(500,)),
    layers.Dense(1024, activation='relu'),
    layers.Dropout(0.1),
    layers.Dense(512, activation='relu'),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.1),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='linear')
])
```



## Pros:

- Common in research - many papers that provide their code will be giving it in PyTorch
- Flexible and powerful once you know how to use it

## Cons:

- Higher barrier to entry – models are written as classes, not as easy to pick up and go especially if you are not a python coder already

```
# Define the network class inheriting from nn.Module
class SimpleDenseNN(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super(SimpleDenseNN, self).__init__()

        # First fully connected (dense) layer
        self.fc1 = nn.Linear(input_dim, hidden_dim)

        # Activation function
        self.relu = nn.ReLU()

        # Second fully connected (dense) layer
        self.fc2 = nn.Linear(hidden_dim, output_dim)
```



# Layer Types

1. **Convolutional Layer:** Extracts features from the input image through convolution operations. Key Parameters: Number of filters, kernel size, strides, padding.
2. **Activation Layer:** Introduces non-linearity to the model, allowing it to learn complex patterns. Common Types: ReLU (Rectified Linear Unit), Sigmoid, Tanh.
3. **Pooling Layer:** Reduces the spatial dimensions (height and width) of the input volume, making the model more computationally efficient and less sensitive to the exact location of features. Types: Max Pooling, Average Pooling.
4. **Batch Normalization Layer:** Normalizes the output of a previous layer by subtracting the batch mean and dividing by the batch standard deviation. Helps in speeding up training and reducing the sensitivity to network initialization.
5. **Dropout Layer:** Randomly sets a fraction of input units to zero at each update during training, helping to prevent overfitting.



THANK YOU!